

Universidad Siglo 21



Trabajo Final de Grado, Prototipado Tecnológico

Carrera: Licenciatura en Informática

Plataforma low-code con inteligencia artificial para la medición y optimización de la huella de carbono en entornos informáticos.

Autor: Recce, Carlos Lautaro

Legajo: VINF013591

Santa Fe, Septiembre de 2025

Índice

Resumen	5
Abstract.....	6
Título	7
Introducción.....	7
Antecedentes	7
Descripción del área problemática.....	9
Justificación.....	10
Objetivo general del proyecto	11
Objetivos específicos del proyecto	11
Marco teórico referencial.....	12
Dominio del problema	12
Tecnología de la información y la comunicación	13
Competencia	15
Diseño metodológico.....	16
Relevamiento	18
Relevamiento estructural	18
Relevamiento funcional	19
Relevamiento de la documentación	22
Proceso de negocio	22
Diagnóstico y Propuesta	23
Objetivo, límites y alcance del prototipo.....	26
Objetivo del prototipo	26
Límites	26
Alcances.....	26

Trabajo Final de Grado – Prototipado Tecnológico

Descripción del sistema	27
Product backlog	27
Historias de usuario	29
Sprint backlog	34
Estructura de datos	38
Prototipos de interfaces de pantallas	40
Prototipo 1: Panel de Operaciones y Telemetría Sostenible	40
Prototipo 2: Pantalla de Gestión de Proyectos	41
Prototipo 3: Pantalla de Recopilación de Requisitos (Green IT)	41
Prototipo 4: Pantalla de Editor y Arquitectura Low-Code	42
Prototipo 5: Pantalla de Historial de Optimizaciones con IA	43
Prototipo 6: Pantalla de Generador de Reportes Ambientales	43
Prototipo 7: Pantalla de Pipeline de CI/CD (Validación y Despliegue)	44
Prototipo 8: Centro de Alertas Ambientales	45
Prototipo 9: Pantalla de Configuración de Plataforma y Seguridad	45
Prototipo 10: Pantalla de Autenticación y Acceso Seguro	46
Encadenamiento Lógico de Prototipos de Interfaces	47
Diagrama de arquitectura	48
Seguridad	49
Análisis de costos	50
Análisis de riesgo	55
Conclusiones	64
Prototipo	65
Referencias	66
Anexos	74

Índice de Figuras

Figura 1 - Diagrama de Gantt	18
Figura 2 - Organigrama	19
Figura 3 - Diagrama de flujo	23
Figura 4 - Diagrama de Entidad-Relación de Plataforma Low-Code	38
Figura 5 - Panel de Operaciones y Telemetría Sostenible.....	40
Figura 6 - Pantalla de Gestión de Proyectos.....	41
Figura 7 - Pantalla de Recopilación de Requisitos (Green IT).....	41
Figura 8 - Pantalla de Editor y Arquitectura Low-Code	42
Figura 9 - Pantalla de Historial de Optimizaciones con IA.....	43
Figura 10 - Pantalla de Generador de Reportes Ambientales.....	43
Figura 11 - Pantalla de Pipeline de CI/CD (Validación y Despliegue).....	44
Figura 12 - Pantalla de Centro de Alertas Ambientales	45
Figura 13 - Pantalla de Configuración de Plataforma y Seguridad	45
Figura 14 - Pantalla de Autenticación y Acceso Seguro	46
Figura 15 - Encadenamiento Lógico de Prototipos de Interfaces.....	47
Figura 16 - Diagrama de arquitectura de Plataforma Low-Code con IA	49
Figura 17 - Diagrama de Pareto.....	62

Índice de Tablas

Tabla 1: Competencias	16
Tabla 2: Tabla para Diagrama de Gantt	18
Tabla 3: Planificación de proyectos de desarrollo de software.	24
Tabla 4: Codificación y desarrollo de aplicaciones	25

Trabajo Final de Grado – Prototipado Tecnológico

Tabla 5: Monitoreo y optimización de recursos post-desarrollo. _____	25
Tabla 6: Product backlog del sistema _____	28
Tabla 7: Recopilación de requisitos iniciales con métricas energéticas. _____	29
Tabla 8: Diseño de arquitectura low-code con soporte IA. _____	30
Tabla 9: Generación de código optimizado mediante IA _____	31
Tabla 10: Ejecución de pruebas de funcionalidad con evaluación energética. _____	32
Tabla 11: Monitoreo de métricas de rendimiento con dashboard IA. _____	32
Tabla 12: Refactorización de código con sugerencias IA. _____	33
Tabla 13: Generación de reportes ambientales automáticos. _____	34
Tabla 14: Despliegue inicial con revisión de métricas ecoeficientes _____	34
Tabla 15: Sprint Backlog HU-001-HU-002-HU-008 _____	37
Tabla 16 - Costos del personal de desarrollo _____	52
Tabla 17 - Costo de operación del personal _____	53
Tabla 18 - Cotización dólar BNA _____	54
Tabla 19 - Costo de hardware y software _____	55
Tabla 20 - Clasificación de riesgos _____	58
Tabla 21 - matriz de riesgos del proyecto. _____	59
Tabla 22 - Matriz de riesgo _____	59
Tabla 23 - Análisis cuantitativo de riesgos _____	60
Tabla 24 - Análisis cuantitativo y grado de exposición _____	62
Tabla 26 - Plan de contingencias _____	64

Resumen

Se desarrolló un prototipo tecnológico de una plataforma de desarrollo low-code integrada con inteligencia artificial, diseñada para optimizar la ecoeficiencia y medir la huella de carbono en el ciclo de vida del software. El sistema se centró en resolver la dispersión de métricas ambientales en el desarrollo tradicional. El prototipo demostró funcionalidades centrales, incluyendo un dashboard que integra métricas de consumo (como CPU y estimaciones de CO₂) en tiempo real, obtenidas de herramientas como CodeCarbon y Climatiq. La plataforma utiliza un entorno visual *drag-and-drop* y emplea IA generativa para analizar y sugerir refactorizaciones automáticas, optimizando el código generado para reducir su consumo energético. El desarrollo se gestionó bajo la metodología SCRUM sobre una arquitectura de nube (AWS) optimizada para sostenibilidad. El prototipo validó la viabilidad de unificar el desarrollo rápido low-code con la monitorización ambiental, demostrando el *core* de una solución para la creación de aplicaciones ecoeficientes.

Palabras clave: plataformas low-code, inteligencia artificial, huella de carbono, software verde.

Abstract

A technological prototype of a low-code development platform integrated with artificial intelligence was developed, designed to optimize eco-efficiency and measure the carbon footprint in the software life cycle. The system focused on solving the dispersion of environmental metrics in traditional development. The prototype demonstrated core functionalities, including a dashboard that integrates consumption metrics (such as CPU and CO2 estimates) in real-time, obtained from tools like CodeCarbon and Climatiq. The platform uses a visual *drag-and-drop* environment and employs generative AI to analyze and suggest automatic refactorizations, optimizing the generated code to reduce its energy consumption. Development was managed under the SCRUM methodology on a cloud architecture (AWS) optimized for sustainability. The prototype validated the feasibility of unifying rapid low-code development with environmental monitoring, demonstrating the *core* of a solution for creating eco-efficient applications.

Keywords: low-code platforms, artificial intelligence, carbon footprint, green software.

Título

Plataforma low-code con inteligencia artificial para la medición y optimización de la huella de carbono en entornos informáticos.

Introducción

En el contexto de la transformación digital acelerada que experimenta la sociedad, el desarrollo de software ha generado un incremento significativo en el consumo energético y las emisiones de carbono asociadas, convirtiéndose en un desafío ambiental crítico. Según informes recientes, el sector tecnológico contribuye con un aumento en las emisiones ambientales, donde cada proceso de desarrollo y ejecución de aplicaciones implica un costo energético que, si no se optimiza, agrava la huella de carbono digital. Además, tendencias para 2025 indican que las empresas priorizarán el desarrollo de software eficiente en consumo energético para reducir esta huella de carbono, alineándose con prácticas de "green coding" que buscan minimizar el impacto ambiental innecesario. Este proyecto se desarrolló en una empresa modelada dedicada a la producción de software y servicios informáticos, donde la dispersión de información sobre métricas de recursos computacionales (como CPU y memoria) y su correlación con emisiones de CO₂ representaba un problema persistente, limitando la capacidad para crear aplicaciones sostenibles. (Kodffe, 2025), (The International Telecommunication Union, 2025).

A lo largo de esta elaboración, se pretendió integrar herramientas de bajo código para la creación de aplicaciones ecoeficientes, democratizando el acceso a optimizaciones automatizadas que reduzcan ciclos ineficientes y emisiones estimadas. Inspirado en predicciones como las de Gartner, que estiman que para 2025 el 70% de las nuevas aplicaciones empresariales utilizarán tecnologías low-code o no-code, el enfoque incorporó IA para analizar y refactorizar código, promoviendo un desarrollo holístico y sostenible que no solo resuelve la disponibilidad dispersa de datos ambientales, sino que contribuye a la mitigación del cambio climático a través de TIC innovadoras. (S&P Global Market Intelligence, 2018), (Seidor, 2025).

Antecedentes

La evolución de las plataformas de desarrollo low-code, definidas como herramientas que permiten la creación de aplicaciones mediante interfaces visuales y

mínimas líneas de código para acelerar el desarrollo (Gartner, 2024), se remonta a inicios del siglo XXI. Por ejemplo, pioneros como OutSystems en 2001 y Mendix en 2005 introdujeron enfoques visuales para acelerar la creación de aplicaciones sin codificación extensiva, democratizando el acceso a la programación para perfiles no expertos (Ninox, 2025). Posteriormente, en la década de 2010, el auge de la transformación digital impulsó su adopción masiva, con predicciones de Gartner en 2018 que estimaban que para 2024, el 65% de las aplicaciones se desarrollarían mediante low-code, democratizando el acceso a herramientas sofisticadas para equipos no expertos (Gartner, 2024). Esta progresión se alineó con la necesidad de metodologías ágiles, incorporando enfoques holísticos y multifuncionales que integran diseño, implementación y monitoreo en un solo entorno, como se observa en plataformas que evolucionaron para soportar DevOps completos, un marco de prácticas que une desarrollo y operaciones para mejorar la eficiencia (S&P Global Market Intelligence, 2018), (Codector, s.f.).

La integración de inteligencia artificial (IA) en plataformas low-code surgió como una tendencia emergente en los años 2020, con desarrollos como copilotos generativos que automatizan la generación y optimización de código, reduciendo tiempos de desarrollo en hasta un 50% según informes de 2024 (Jitterbit, 2025). Ejemplos incluyen Microsoft Power Platform, que combina IA para procesamiento de datos y toma de decisiones, y herramientas como GeneXus que utilizan IA generativa para asistir en la automatización de procesos (Outsystems, La plataforma líder de desarrollo de código bajo impulsada por IA, 2025a). Esta convergencia se extendió hacia la sostenibilidad, donde la IA se aplica para optimizar recursos y minimizar impactos ambientales, respondiendo a problemas como la dispersión de métricas de consumo energético en el desarrollo de software.

La Green Software Foundation, establecida en 2021, ha proporcionado marcos para medir y reducir emisiones, integrando principios como la optimización de algoritmos y el uso de infraestructuras ecológicas (Green Software Foundation, 2023). En el contexto de la empresa modelada EcoDev Tech, fundada en 2018 como proveedora de soluciones de software en Argentina, la evolución incluyó una transición hacia prácticas sostenibles en 2022, motivada por la creciente dispersión de métricas ambientales en sus procesos de desarrollo, lo que sirvió de base para abordar esta problemática a través de innovaciones en low-code e IA (BBVA, 2023; IBM, 2023).

En paralelo, el concepto de green coding, que se refiere a prácticas de programación que buscan minimizar la huella de carbono del software mediante eficiencia energética (E-SPIN Group, 2025), emergió en respuesta al impacto ambiental del sector IT, que representa aproximadamente el 2-3% de las emisiones globales de CO₂ según datos de 2023, impulsando prácticas para desarrollar software energéticamente eficiente desde la planificación hasta la ejecución (CIO, 2024; Cloudwards, 2025). Esta tendencia no solo reduce emisiones, sino que aborda problemas como el aumento de centros de datos intensivos en energía, alineándose con regulaciones ambientales y preparando el terreno para proyectos como el propuesto, donde se integra low-code con IA para optimizar recursos y visualizar impactos en tiempo real.

Descripción del área problemática

En el contexto de la empresa modelada, se identificó que el área problemática principal se centraba en el desarrollo de software, donde la falta de integración entre métricas de consumo energético y prácticas sostenibles generaba ineficiencias ambientales persistentes. Las áreas afectadas incluían los procesos de codificación, pruebas y optimización de aplicaciones, que impactaban directamente en el uso de recursos computacionales como CPU y memoria, extendiéndose a la infraestructura de centros de datos. Los factores involucrados abarcaban la dispersión de información sobre emisiones de CO₂ asociadas al código, la ausencia de herramientas automatizadas para refactorización ecológica y el crecimiento exponencial de cargas de trabajo impulsadas por IA, lo que exacerbaba el consumo innecesario de energía (Mediaware, 2025; The International Telecommunication Union, 2025; Jens Gröger, 2025).

La magnitud del problema era significativa: el sector de centros de datos, esencial para el desarrollo de software, representaba cerca del 2% del consumo eléctrico mundial, emitiendo hasta 50.000 toneladas de CO₂ anuales (equivalente a las emisiones de 10.000 automóviles). Además, el entrenamiento de modelos de IA en plataformas de desarrollo podía generar hasta 300 toneladas de CO₂ por modelo grande, y ciertos comandos en IA multiplicaban las emisiones hasta 50 veces dependiendo de la tarea. En nuestra empresa, esta magnitud se manifestaba en proyectos donde el consumo energético no optimizado elevaba costos operativos y ambientales, alineándose con proyecciones globales que indicaban un aumento del 150% en emisiones indirectas de carbono por el uso de IA en empresas tecnológicas (Holdfast Projects, 2025; Crownhart, 2025; Poidevin, 2025).

La frecuencia de presentación del problema era constante y diaria, ocurriendo en cada ciclo de desarrollo de aplicaciones, desde la ideación hasta la implementación, lo que acumulaba impactos ambientales a lo largo de múltiples iteraciones en equipos multifuncionales. Las tendencias observadas apuntaban a un incremento en la demanda energética, con estimaciones de que los centros de datos podrían representar el 4% del consumo mundial de electricidad para 2030, impulsado por la expansión de IA y software. En paralelo, se informa una tendencia hacia la adopción de plataformas low-code sostenibles, con predicciones de que el 70% de las nuevas aplicaciones empresariales se desarrollarían con estas tecnologías para 2025, priorizando eficiencia energética y green coding para minimizar huellas de carbono (Antonio Romero y Craig Hale, 2024; Jens Gröger, 2025).

Las personas implicadas incluían desarrolladores profesionales, analistas de negocios y arquitectos de aplicaciones de la empresa modelada, mientras que las instituciones involucradas abarcaban organizaciones como la Green Software Foundation y empresas tecnológicas globales como Google y Microsoft, cuyas emisiones totales habían aumentado un 23.4% desde 2020 debido a factores similares, destacando la necesidad de intervenciones en green IT (News Center Microsoft Latinoamérica, 2025), (Fernanda González, 2024).

Justificación

Este proyecto se justificó por la necesidad urgente de herramientas que abordaran la huella ambiental del software en la era de la transformación digital, donde el desarrollo tradicional generaba un alto consumo energético innecesario, contribuyendo al 3-4% de las emisiones globales de CO₂ según estimaciones de 2025. Al llevar adelante este prototipado tecnológico, contribuí a la democratización de prácticas sostenibles, permitiendo a empresas y desarrolladores crear aplicaciones ecoeficientes, alineado con predicciones de Gartner de que para 2025, más del 70% de las nuevas aplicaciones empresariales se desarrollarían con tecnologías low-code o no-code, priorizando la optimización de recursos para green IT. Esto resolvió problemas reales como la dispersión de métricas de recursos computacionales y emisiones de CO₂, promoviendo una innovación que integraba IA para eficiencia ambiental en un contexto de creciente presión regulatoria. (TsTTechnology, 2025)

Las necesidades cubiertas incluyeron la optimización energética en procesos de desarrollo, reduciendo ineficiencias como ciclos innecesarios en código que elevaban el consumo en centros de datos, lo que podía generar hasta 300 toneladas de CO2 por modelo de IA grande. El impacto en el campo tecnológico fue significativo, al extender plataformas low-code hacia la sostenibilidad mediante IA generativa para refactorización automática, fomentando un enfoque holístico que minimizaba el uso de recursos y alineaba con tendencias de 2025 hacia entornos de trabajo ligeros y algoritmos eficientes. Para la organización modelada, el valor radicó en la reducción de costos energéticos y operativos, el cumplimiento de objetivos de sostenibilidad que mejoraban su reputación en el mercado, y la generación de ventajas competitivas en un sector donde el green coding impulsaba ingresos mediante eficiencia y atracción de talento. (Bioscore, 2024).

La innovación produjo avances en procesos de desarrollo al incorporar dashboards para visualización en tiempo real de métricas ambientales, resolviendo la falta de integración en herramientas tradicionales y promoviendo una utilidad factible para equipos multifuncionales. Esto generó interés societal al mitigar el cambio climático a través de TIC, con potencial para reducir emisiones indirectas en hasta un 85% en productos de software, contribuyendo a un futuro digital más verde y accesible. (Techtalent, 2025)

Objetivo general del proyecto

Promover la sostenibilidad ambiental en las empresas de desarrollo de software mediante la implementación de una plataforma low-code integrada con inteligencia artificial que optimice la creación de aplicaciones ecoeficientes, reduciendo el consumo energético y las emisiones de CO2 en un 70% respecto a métodos tradicionales, evaluado a través de métricas comparativas en entornos informáticos reales.

Objetivos específicos del proyecto

- Investigar y seleccionar entornos de trabajo low-code y modelos de inteligencia artificial compatibles con métricas ambientales para optimizar el consumo energético en aplicaciones de software.

- Diseñar una interfaz drag-and-drop que permitiera a usuarios no expertos configurar aplicaciones simples con componentes para reducir ineficiencias computacionales.
- Integrar algoritmos de inteligencia artificial que analizaran el código generado y propusieran refactorizaciones automáticas para minimizar el uso de recursos como CPU y memoria.
- Desarrollar un dashboard integrado que visualizara en tiempo real métricas de huella de carbono y emisiones de CO₂, resolviendo la dispersión de información ambiental.
- Evaluar la eficiencia energética del prototipo mediante pruebas controladas que midieran reducciones en emisiones estimadas, validando un impacto ambiental menor en un 70% comparado con métodos tradicionales.

Marco teórico referencial

Dominio del problema

Para comprender el dominio del problema, se definió una plataforma low-code como un entorno de desarrollo que permite la creación de aplicaciones mediante interfaces visuales y configuraciones mínimas de código, enfocándose en la democratización del desarrollo para reducir tiempos y costos. Según Gartner (2025), estas plataformas apoyan la composición de aplicaciones empresariales, integrando capacidades de automatización y escalabilidad para adaptarse a necesidades complejas en entornos digitales. En el contexto de la sostenibilidad y green coding se conceptualizó como un conjunto de prácticas que minimizan el consumo energético y de recursos a lo largo del ciclo de vida del software, desde el diseño hasta el mantenimiento, promoviendo algoritmos eficientes y optimizaciones que reducen la huella de carbono. Esta aproximación se alineó con regulaciones ambientales emergentes, donde el sector IT contribuye significativamente a emisiones globales, impulsando la necesidad de integrar métricas ambientales en procesos de desarrollo para lograr una reducción cuantificable en el impacto ecológico (Meghan Rimol DeLisi y Catherine Howley, 2022).

Además, la inteligencia artificial en el desarrollo sostenible se entendió como el uso de modelos algorítmicos para automatizar optimizaciones, tales como la refactorización de código para eficiencia energética, resolviendo ineficiencias que

generan emisiones innecesarias en centros de datos. El problema de la dispersión de información sobre métricas ambientales, como el consumo de CPU y emisiones de CO₂, se identificó como una barrera clave, donde herramientas como el Software Carbon Intensity (SCI) proporcionan marcos para medir y mitigar estas emisiones de manera estandarizada. Esto permitió reconocer las necesidades del negocio en EcoDev Tech, enfocadas en alinear el desarrollo de software con objetivos de sostenibilidad para cumplir normativas como las del GHG Protocol, que guían la medición de emisiones en operaciones digitales. (Meghan Rimol DeLisi y Catherine Howley, 2022), (Superblocks Team, 2025).

Tecnología de la información y la comunicación

Para el desarrollo del proyecto, se exploraron tecnologías clave que soportaran la integración de low-code con IA y métricas ambientales. Estas se detallaron a continuación:

- Entornos de trabajo low-code: Frameworks (como OutSystems y Mendix) se seleccionaron por su capacidad para acelerar el desarrollo mediante entornos visuales, permitiendo la creación de aplicaciones full-stack con integración nativa de IA para automatización. OutSystems, por ejemplo, ofrece herramientas de alto rendimiento que soportan ciclos DevOps completos, facilitando la optimización energética mediante componentes reutilizables. OutSystems se destaca por su enfoque en el desarrollo impulsado por IA, que permite la creación de agentes personalizados y aplicaciones empresariales con gobernanza integral del ciclo de vida del software, incorporando prácticas de codificación segura y eficiente que reducen la huella de carbono al minimizar recursos computacionales innecesarios y promover despliegues escalables en entornos de nube optimizados para sostenibilidad. Estos entornos de trabajo incluyen características de seguridad integradas, como autenticación multifactor (MFA), control de acceso basado en roles similar a IAM, y análisis de patrones de comportamiento mediante IA para detectar accesos anómalos, alineándose con estándares para prevenir intrusiones en plataformas low-code (Processa, 2025; Gartner, 2025). Estos entornos se alinearon con predicciones de Gartner para 2025, donde el 70% de aplicaciones empresariales incorporan low-code para resiliencia y agilidad (Outsystems, 2025a).

- Modelos de inteligencia artificial: Modelos de IA generativa, como los integrados en plataformas (Appsmith AI y Microsoft Power Apps), se utilizaron para

analizar y refactorizar código automáticamente, reduciendo ineficiencias energéticas. Estos modelos emplean técnicas de machine learning para sugerir optimizaciones, como la minimización de bucles innecesarios, integrándose con APIs para cálculos en tiempo real (Appsmith, 18 prácticas recomendadas para desarrollar aplicaciones web seguras y de alto rendimiento con poco código, 2025; Microsoft, 2025).

- Herramientas para medición de huella de carbono: Herramientas como Cloud Carbon Footprint y CodeCarbon se incorporaron para estimar emisiones de CO2 asociadas al código, midiendo el impacto en infraestructuras nube y proporcionando dashboards para visualización. ClimaTiq, con su API para cálculos automatizados, permitió integrar métricas precisas basadas en datos regionales de energía (Footprint, s.f.; ClimaTiq, 2023; Lottick, 2019).

- Lenguajes y entornos de programación: JavaScript se empleó para el frontend debido a su compatibilidad con interfaces drag-and-drop, mientras que Python respaldó los backends de IA por su eficiencia en bibliotecas. Estos se complementaron con entornos de nube como AWS, optimizados para bajo consumo mediante prácticas green. En AWS, se utilizaron servicios como S3 para almacenamiento seguro con encriptación AES-256 en reposo y en tránsito, respaldos automatizados con periodicidad diaria completa e incremental horaria para un RPO menor a 1 hora, geo-replicación para redundancia, y IAM para control de acceso restringido con auditoría de logs; además, se implementaron estrategias de alta disponibilidad 24/7 mediante clústeres redundantes, balanceo de carga automático y replicación multi región, asegurando un tiempo de actividad superior al 99,99% con monitoreo impulsado por IA para mitigar fallos de manera eco-eficiente (Paull Young, 2024; Ugarte Dessire, 2024; Amazon Web Services, 2025a; Amazon Web Services, 2025b; Wiz, 2025; Kolla, 2025).

- Tecnologías de seguridad y despliegue en la nube: Para garantizar la integridad y confidencialidad de la plataforma y los datos generados, se emplean tecnologías estándar de la industria.

- Autenticación Multifactor: Es un método de seguridad que requiere que los usuarios proporcionen dos o más factores de verificación para acceder a un recurso. Estos factores suelen ser una combinación de algo que el usuario sabe (contraseña), algo que tiene (un token o teléfono) y algo que es (biometría). Su

implementación es una práctica recomendada por estándares como OWASP para mitigar riesgos de suplantación de identidad (OWASP, 2025).

- Amazon Web Services S3 y AES-256: AWS S3 (Simple Storage Service) es un servicio de almacenamiento de objetos en la nube que ofrece alta durabilidad y escalabilidad. Para la protección de datos, se utiliza el Estándar de Cifrado Avanzado (AES) con claves de 256 bits (AES-256), considerado el estándar de la industria para el cifrado de datos tanto en reposo (almacenados en disco) como en tránsito (mientras se mueven por la red) (Amazon Web Services, 2025a).

- AWS IAM: Es el servicio de AWS que permite gestionar de forma segura el acceso a los servicios y recursos de la nube. IAM (Identity and Access Management) posibilita la creación de usuarios y roles, aplicando el principio de mínimo privilegio, que consiste en otorgar únicamente los permisos necesarios para que una entidad (usuario o servicio) realice su tarea específica (Wiz, 2025).

- Alta Disponibilidad y Geo-replicación: La alta disponibilidad se refiere al diseño de sistemas que garantiza un alto nivel de rendimiento operativo, generalmente medido como un porcentaje de tiempo de actividad (ej. 99,99%). Esto se logra mediante la redundancia de componentes (clústeres, balanceo de carga) y la geo-replicación, que consiste en almacenar copias de los datos o la infraestructura en ubicaciones geográficas distintas para asegurar la continuidad del negocio ante fallos regionales (Scale Computing, 2025).

Competencia

Se analizaron plataformas low-code existentes para identificar fortalezas y debilidades en integración de IA, soporte a sostenibilidad y escalabilidad, sin incluir el prototipo propuesto. La comparación se resumió en la siguiente tabla, basada en revisiones de características oficiales y evaluaciones de escalabilidad mediante métricas como tiempo de despliegue, manejo de usuarios concurrentes (hasta 10.000 para empresas) y reducción de emisiones vía optimizaciones AI (medidas en evaluaciones de rendimiento de consumo energético, como kWh por aplicación desplegada).

Plataforma	Integración de IA	Sostenibilidad	Escalabilidad
OutSystems	AI para automatización y optimización	Optimizaciones energéticas básicas	Enterprise full-stack
Mendix	Colaboración con Siemens AI	Enfoque en green IT	Desarrollo visual ágil
Microsoft Power Apps	Copilot para generación de código	Integración con Azure green tools	Integración Microsoft ecosystem
Bubble	AI para sugerencias básicas	Sin métricas ambientales nativas	Apps web escalables

Tabla 1: Competencias

Fuente: Elaboración propia basada en análisis de plataformas (Bubble, 2025) (Mendix, Mendix: Low-Code Application Development Platform, 2025) (Microsoft, 2025).

Diseño metodológico

El proceso para desarrollar el proyecto siguió los lineamientos de la metodología ágil Scrum, que permitió iteraciones cortas y colaborativas para integrar componentes low-code con IA, enfocándose en entregas incrementales que validaran la optimización energética y la sostenibilidad ambiental de manera progresiva. Esta aproximación apuntó a la flexibilidad en el manejo de requisitos emergentes, como ajustes en algoritmos de IA basados en métricas de CO2, alineándose con los objetivos específicos de investigación, diseño e integración para asegurar un prototipo funcional y ecoeficiente.

En cuanto a las herramientas de desarrollo, se utilizó entornos de trabajo low-code como OutSystems para la interfaz drag-and-drop y generación de código base, integrados con modelos de IA en Python mediante bibliotecas para el análisis y refactorización automática, lo que permitió optimizar el consumo de recursos computacionales en un flujo unificado (Outsystems, 2025a; Outsystems, 2025b). Estas se complementaron con

herramientas de medición para estimar emisiones de CO2 en tiempo real, enlazadas a través de APIs, demostrable mediante paquetes como CodeCarbon que estiman emisiones durante la ejecución de código, mostrando reducciones en evaluaciones de rendimiento de optimización (Lottick, 2019). JavaScript manejó el frontend para interfaces intuitivas, uniendo todos los elementos en un sistema holístico que minimizó ineficiencias energéticas durante las iteraciones.

Para la recolección de datos, se aplicó técnicas de entrevistas y análisis y estudios de documentación interna en la empresa modelada, lo que facilitó la comprensión completa de procesos actuales de desarrollo y problemas de dispersión de métricas ambientales. Las entrevistas se dirigieron a roles como el gerente de desarrollo de software y un desarrollador senior, con preguntas enfocadas en flujos de trabajo, consumo energético y necesidades de optimización. La estructura de estas entrevistas se detalla en el Anexo 1.

La planificación del proyecto se detalló en la siguiente tabla de planificación y el diagrama de Gantt:

Tarea	Duración	Fecha Inicio	Fecha Fin
Inicio del Seminario Final de Informática	-	4/8/25	-
Elección del tema	7 días	4/8/25	10/8/25
Título	1 día	11/8/25	11/8/25
Introducción	2 días	12/8/25	13/8/25
Justificación	2 días	14/8/25	15/8/25
Objetivo general del proyecto	3 días	16/8/25	18/8/25
Objetivos específicos del proyecto	3 días	19/8/25	21/8/25
Marco teórico referencial	7 días	22/8/25	28/8/25
Diseño metodológico	6 días	29/8/25	3/9/25
Relevamiento	7 días	4/9/25	10/9/25
Procesos de negocio	4 días	11/9/25	14/9/25
Diagnóstico y propuesta	5 días	15/9/25	19/9/25
Objetivo, límites y alcance del prototipo	6 días	20/9/25	25/9/25
Descripción del sistema	10 días	26/9/25	5/10/25
Seguridad	6 días	6/10/25	11/10/25
Análisis de costos	5 días	12/10/25	16/10/25
Análisis de riesgo	7 días	17/10/25	23/10/25
Conclusiones	3 días	24/10/25	26/10/25
Demo	21 días	27/10/25	16/11/25

Tabla 2: Tabla para Diagrama de Gantt

Fuente: Elaboración propia en Word.

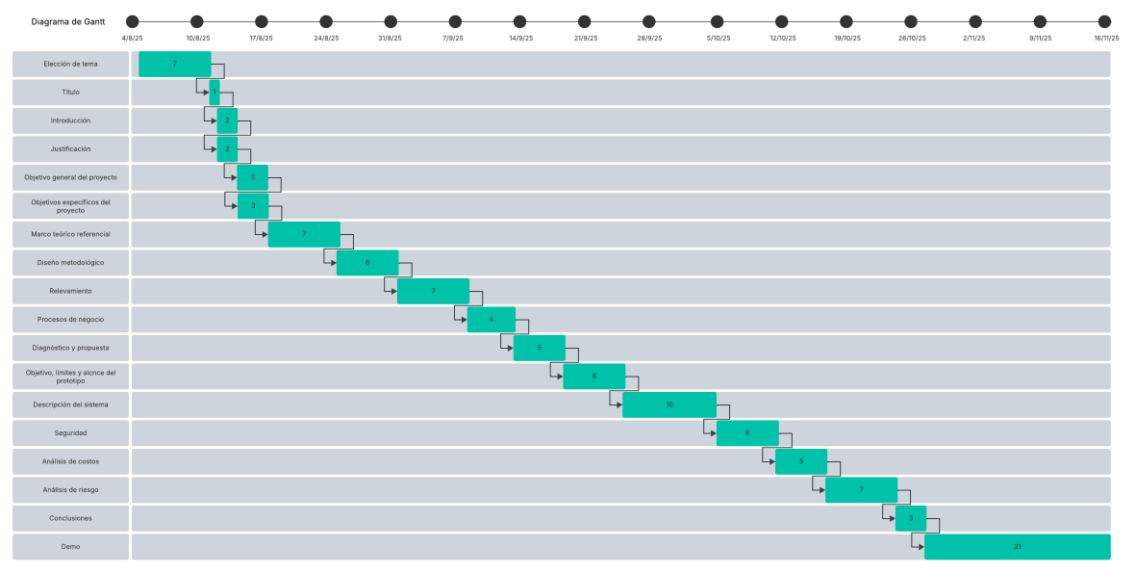


Figura 1 - Diagrama de Gantt

Fuente: Elaboración propia en Lucidchart.

Relevamiento

Relevamiento estructural

Al tratarse de una organización modelada ficticia, la empresa se conceptualizó como una empresa dedicada al desarrollo de software y servicios informáticos en Argentina, con operaciones distribuidas en un entorno de nube híbrido para facilitar la escalabilidad y el acceso remoto. El marco de implementación del proyecto se destinó a un data center local virtualizado, donde los procesos de desarrollo se ejecutaban en servidores compartidos con proveedores como AWS, permitiendo la integración de herramientas de monitoreo ambiental sin infraestructura física específica preexistente (Amazon Web Services, 2025a). Esto permitió simular un contexto donde la dispersión de métricas energéticas se manejaba manualmente a través de herramientas dispersas, como hojas de cálculo y APIs externas, sin un sistema unificado, basado en evaluaciones que estiman consumos mediante métricas como kWh por carga de trabajo, midiendo dispersiones típicas en entornos híbridos de hasta 30% en ineficiencia energética (Amazon Web Services, 2025b).

Relevamiento funcional

Para el relevamiento funcional, se modeló la estructura jerárquica basándose en organizaciones equivalentes del sector de desarrollo de software, adaptando modelos estándar de empresas tecnológicas medianas. Esto facilitó la identificación de funciones y procesos actuales relacionados con el desarrollo de aplicaciones, donde la falta de optimización energética era evidente.

La estructura jerárquica de la empresa se representó mediante un organigrama que reflejaba una organización típica de software, con énfasis en las áreas involucradas en el ciclo de desarrollo. En la Figura 2 (elaboración propia, adaptada de modelos estándar en el sector IT), se mostró la gerencia general al tope, ramificándose en departamentos clave.

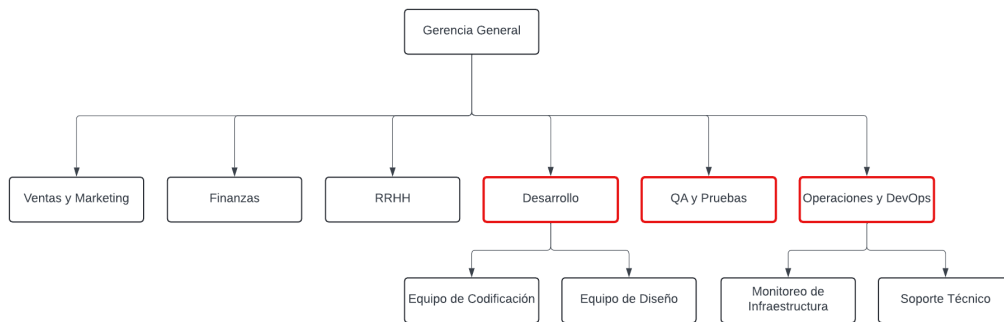


Figura 2 - Organigrama

Fuente: Elaboración propia en Lucidchart

Las funciones de las áreas involucradas se describen a continuación:

- Desarrollo: Se encargaba de la investigación inicial de requisitos, el diseño de arquitectura y la codificación manual de aplicaciones, incluyendo la integración de componentes reutilizables sin consideraciones automáticas de eficiencia energética.

- QA y Pruebas: Realizaba evaluaciones de funcionalidad y rendimiento, identificando ineficiencias en código pero sin herramientas integradas para métricas ambientales, lo que requería revisiones manuales.

- Operaciones y DevOps: Gestionaba el despliegue inicial, el monitoreo de recursos computacionales y la generación de reportes dispersos sobre consumo, utilizando herramientas externas para estimaciones básicas de CO2.

A partir de las técnicas de recolección de datos aplicadas, relevamos los siguientes procesos actuales, con aproximadamente 8 identificados para cubrir el ciclo de desarrollo

completo. Cada proceso se detalló con su nombre actual, roles involucrados y pasos secuenciales, enfocados en cómo se ejecutaban sin alusiones a mejoras futuras.

- Proceso 1: Investigación de requisitos.
- Roles: Gerente de Proyecto, Analista de Negocios, Desarrollador Senior.
- Pasos: El gerente de proyecto iniciaba la recopilación de necesidades del cliente mediante reuniones virtuales; el analista de negocios documentaba los requisitos en hojas de cálculo compartidas; el desarrollador senior validaba la factibilidad técnica inicial sin integrar métricas energéticas, y se archivaba la documentación en un repositorio compartido.
- Proceso 2: Diseño de arquitectura.
- Roles: Arquitecto de Software, Equipo de Diseño, Gerente de Proyecto.
- Pasos: El arquitecto de software esbozaba la estructura de la aplicación en diagramas manuales; el equipo de diseño seleccionaba componentes reutilizables de bibliotecas existentes; el gerente de proyecto aprobaba el diseño preliminar, considerando solo aspectos funcionales sin análisis de impacto ambiental, y se compartía el borrador vía email para retroalimentación.
- Proceso 3: Codificación manual.
- Roles: Desarrollador Junior, Desarrollador Senior, Equipo de Codificación.
- Pasos: El desarrollador junior escribía código base en editores estándar; el desarrollador senior revisaba y agregaba lógica compleja, incluyendo llamadas a APIs externas; el equipo de codificación compilaba y depuraba manualmente, ignorando optimizaciones energéticas a menos que se detectaran errores evidentes, y se versionaba en Git sin métricas asociadas.
- Proceso 4: Pruebas de funcionalidad.
- Roles: Analista de QA, Tester Automatizado, QA Lead.
- Pasos: El analista de QA ejecutaba casos de prueba manuales en entornos locales; el tester automatizado corría scripts básicos para validar flujos; el QA lead compilaba resultados en reportes dispersos, enfocándose en errores funcionales sin medir consumo de recursos durante las ejecuciones, y se archivaban los logs en carpetas compartidas.
- Proceso 5: Monitoreo de rendimiento.

- Roles: Ingeniero de Operaciones, DevOps Especialista, Monitoreador de Infraestructura.
- Pasos 6: El ingeniero de operaciones configuraba herramientas básicas como New Relic para métricas de CPU y memoria; el DevOps especialista revisaba logs manualmente en intervalos diarios; el monitoreador de infraestructura generaba reportes ad hoc sobre picos de consumo, sin correlación automática con emisiones de CO2, y se distribuían vía email a los equipos relevantes.
- Proceso 7: Optimización manual de código.
- Roles: Desarrollador Senior, Arquitecto de Software, Equipo de Desarrollo.
- Pasos: El desarrollador senior identificaba ineficiencias en revisiones de código peer-to-peer; el arquitecto de software sugería refactorizaciones basadas en experiencia; el equipo de desarrollo aplicaba cambios iterativos, midiendo impactos manualmente con herramientas como analizadores de rendimiento, y se documentaba el proceso en notas internas sin dashboards integrados.
- Proceso 8: Generación de reportes ambientales.
- Roles: Analista de Sostenibilidad, Gerente de Operaciones, Equipo de DevOps.
- Pasos: El analista de sostenibilidad recopilaba datos dispersos de múltiples fuentes como APIs de carbono; el gerente de operaciones validaba estimaciones manuales de CO2; el equipo de DevOps agregaba notas sobre huella energética en reportes mensuales, utilizando calculadoras externas sin integración real-time, y se almacenaban en drives compartidos.
- Proceso 9: Despliegue inicial y revisión.
- Roles: DevOps Especialista, Gerente de Proyecto, Soporte Técnico.
- Pasos: El DevOps especialista preparaba el entorno de nube para despliegue; el gerente de proyecto aprobaba la versión basada en pruebas previas; el soporte técnico monitoreaba el lanzamiento inicial, detectando problemas posdespliegue manualmente sin alertas automáticas para ineficiencias energéticas, y se registraba el feedback en tickets internos.

Relevamiento de la documentación

Durante el relevamiento, se identificaron documentos clave mencionados en los procesos, que se respaldaron en anexos para ilustrar los flujos actuales:

- Hoja de requisitos inicial (Anexo 2): Documento en formato Excel utilizado para registrar necesidades del cliente, incluyendo especificaciones funcionales sin métricas ambientales.
- Reporte de pruebas de QA (Anexo 3): Formato PDF con logs de ejecuciones, detallando errores, pero no consumos energéticos.
- Log de monitoreo de recursos (Anexo 4): Archivo de texto con métricas dispersas de CPU y memoria, generado manualmente.
- Estimación de CO2 manual (Anexo 5): Hoja de cálculo para cálculos ad hoc de emisiones, basada en herramientas externas.

Proceso de negocio

Para el modelado del proceso de negocio, empleé un diagrama de flujo, que integró los procesos relevados en el relevamiento funcional para representar el ciclo genérico de desarrollo de software en la empresa. Este proceso genérico, denominado "Ciclo de Desarrollo de Aplicaciones", enfocó en el core del negocio: la secuencia desde la investigación de requisitos hasta el despliegue inicial y revisión, incorporando interacciones entre codificación, pruebas, monitoreo y optimización manual, con énfasis en la gestión dispersa de métricas energéticas y ambientales. No se incluyeron procesos secundarios como ventas o recursos humanos, ya que no formaban parte del núcleo relevado.

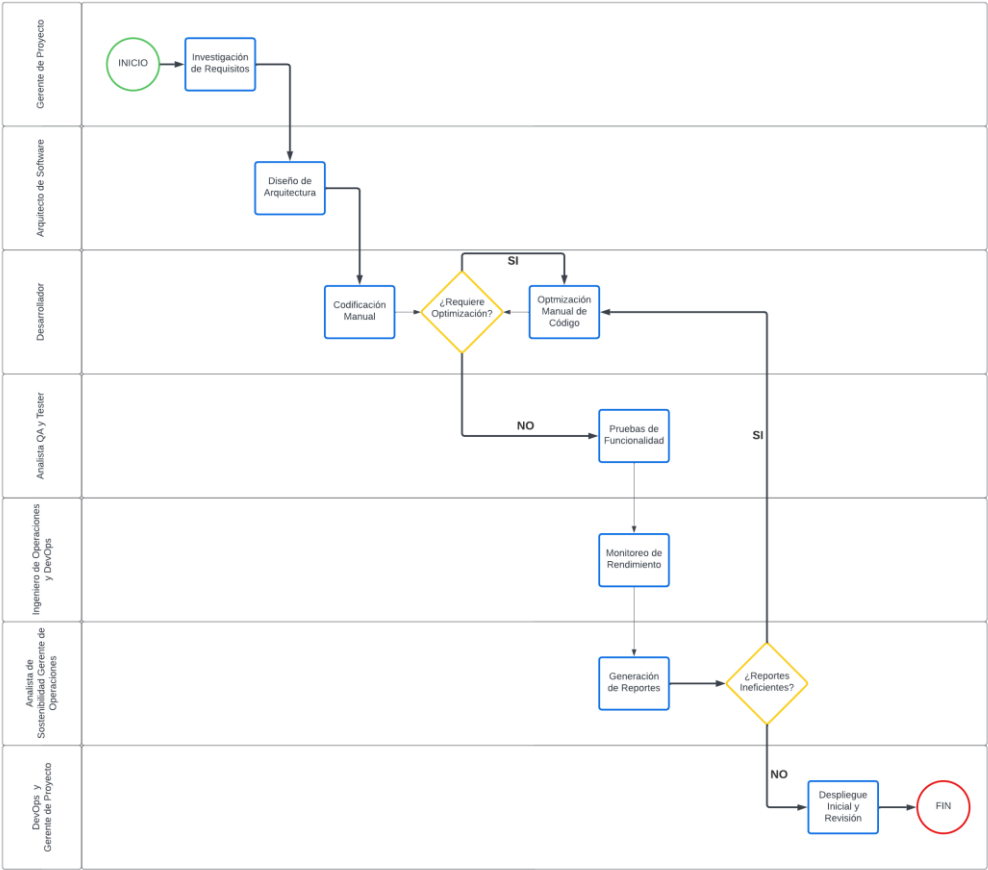


Figura 3 - Diagrama de flujo

Fuente: Elaboración propia en Lucidchart.

Diagnóstico y Propuesta

En esta sección se muestran aquellos procesos que impactan principalmente en el proyecto, seleccionados del relevamiento funcional realizado en la organización modelada. Estos procesos se diagnosticaron con base en los inconvenientes detectados en el manejo de información y recursos durante el desarrollo de software tradicional, sin incluir aquellos que funcionaban de manera adecuada. El diagnóstico se centra en problemas relacionados con la ineficiencia energética y la sostenibilidad, derivados de prácticas convencionales en entornos informáticos.

Nombre del proceso: Planificación de proyectos de desarrollo de software	
Problemas	Causas
1. Se observaron retrasos frecuentes en la estimación de recursos	1. La ausencia de métricas históricas integradas en las herramientas

computacionales necesarios para el proyecto, lo que generaba sobreasignación de hardware y mayor consumo energético.	de planificación manuales impedía una evaluación precisa de patrones de uso previos (Pereira, 2020). 2. La dependencia de hojas de cálculo y documentos dispersos dificultaba la consolidación de datos en tiempo real, lo que prolongaba las revisiones iterativas.
2. La falta de integración entre requisitos funcionales y consideraciones ambientales resultaba en planes que no priorizaban la ecoeficiencia desde el inicio.	1. Los equipos multidisciplinarios no contaban con protocolos estandarizados para incorporar análisis de impacto energético en las fases iniciales, lo que derivaba en omisiones sistemáticas (Piattini, 2015).

Tabla 3: Planificación de proyectos de desarrollo de software.

Fuente: Elaboración propia.

Nombre del proceso: Codificación y desarrollo de aplicaciones	
Problemas	Causas
1. El código generado presentaba ciclos redundantes y algoritmos no optimizados, incrementando el consumo de CPU y memoria durante la ejecución.	1. La codificación manual sin guías automatizadas fomentaba la inclusión de estructuras ineficientes, como bucles anidados innecesarios, debido a la variabilidad en las habilidades de los desarrolladores (Pereira, 2020). 2. La ausencia de revisiones colaborativas en tiempo real propiciaba la acumulación de código obsoleto, que requería más procesamiento para su mantenimiento.

2. Se detectó una dispersión en el uso de lenguajes de programación, lo que impedía la estandarización de prácticas ecoeficientes.	1. La selección de lenguajes se basaba en preferencias individuales en lugar de criterios de eficiencia energética, como el menor uso de recursos en compilación y tiempo de ejecución (Georgiou, 2019).
--	--

Tabla 4: Codificación y desarrollo de aplicaciones

Fuente: Elaboración propia.

Nombre del proceso: Monitoreo y optimización de recursos post-desarrollo	
Problemas	Causas
1. El seguimiento del consumo energético en aplicaciones desplegadas era inconsistente, lo que dificultaba la identificación de picos de ineficiencia ambiental.	1. Las herramientas de monitoreo manuales no proporcionaban visualizaciones integradas, requiriendo análisis posteriores que retrasaban las correcciones (Piattini, 2015). 2. La falta de integración entre datos de tiempo de ejecución y métricas ambientales generaba informes fragmentados, impidiendo una visión holística.
2. Las refactorizaciones para reducir la huella de carbono se realizaban de forma reactiva, en lugar de proactiva, aumentando el impacto acumulado.	1. Los procesos de revisión dependían de auditorías periódicas en lugar de alertas continuas, lo que permitía la persistencia de ineficiencias (Georgiou, 2019).

Tabla 5: Monitoreo y optimización de recursos post-desarrollo.

Fuente: Elaboración propia.

En respuesta a los problemas diagnosticados, se propuso el desarrollo de una plataforma low-code integrada con inteligencia artificial, diseñada para facilitar la creación de aplicaciones ecoeficientes en entornos informáticos. Esta solución permitía

la automatización de optimizaciones de código, la visualización en tiempo real de métricas de consumo energético y la refactorización inteligente para minimizar emisiones de CO₂, alineándose con tendencias emergentes en green IT. De esta manera, se abordaban las ineficiencias identificadas mediante un enfoque holístico que generalizaba el desarrollo sostenible, extendiendo las capacidades de plataformas low-code tradicionales hacia la sostenibilidad ambiental (Outsystems, La plataforma líder de desarrollo de código bajo impulsada por IA, 2025a; Mendix, Mendix: Low-Code Application Development Platform, 2025).

Objetivo, límites y alcance del prototipo

Objetivo del prototipo

Desarrollar un prototipo de sistema que permita la integración de inteligencia artificial en una plataforma low-code para optimizar la creación de aplicaciones ecoeficientes, reduciendo el consumo energético y las emisiones de CO₂ en entornos informáticos.

Límites

El límite del prototipo tecnológico se definió desde la definición inicial de requisitos para la creación de una aplicación hasta la generación optimizada del código eco-eficiente con métricas de impacto ambiental visualizadas en el dashboard.

Alcances

El alcance del prototipo se delimitó a los procesos de negocio identificados dentro del límite establecido, los cuales se mencionan a continuación en términos generales para garantizar la demostración del core del proyecto enfocado en la optimización eco-eficiente mediante IA en una plataforma low-code:

- Investigación de requisitos.
- Diseño de arquitectura.
- Codificación manual.
- Pruebas de funcionalidad.
- Monitoreo de rendimiento.
- Optimización manual de código.
- Generación de reportes ambientales.

- Despliegue inicial y revisión.

Descripción del sistema

Product backlog

El product backlog se elaboró como un listado ordenado de requisitos del sistema para el prototipo, identificando el contenido descriptivo de cada historia de usuario, la priorización, los puntos de historia y las dependencias. Este backlog se enfocó en los requisitos funcionales y no funcionales relevantes al core del proyecto, alineados con la metodología SCRUM.

ID	Historia de usuario	Prioridad	Puntos de historia	Dependencias
HU-001	Como gerente de proyecto, se requiere recopilar requisitos iniciales con integración de métricas energéticas para planificar aplicaciones ecoeficientes.	Alta	13	
HU-002	Como arquitecto de software, se necesita diseñar la arquitectura low-code con soporte IA para seleccionar componentes reutilizables que minimicen el impacto ambiental.	Alta	8	HU-001
HU-003	Como desarrollador, se busca generar código optimizado automáticamente mediante IA para reducir ciclos redundantes y consumo de CPU en aplicaciones web/móviles.	Media	13	HU-002

HU-004	Como analista de QA, se debe ejecutar pruebas de funcionalidad con evaluación de eficiencia energética para validar el rendimiento eco-eficiente.	Media	5	HU-003
HU-005	Como ingeniero de operaciones, se requiere monitorear métricas de rendimiento en tiempo real con dashboard IA para detectar picos de consumo energético.	Baja	8	HU-004
HU-006	Como desarrollador senior, se necesita refactorizar código manualmente con sugerencias IA para optimizar huella de carbono post-generación.	Baja	5	HU-003
HU-007	Como analista de sostenibilidad, se busca generar reportes ambientales automáticos con estimaciones de CO2 para comparar con métodos tradicionales.	Media	8	HU-005
HU-008	Como DevOps especialista, se requiere desplegar la aplicación inicial con revisión de métricas ecoeficientes para asegurar alineación con objetivos de sostenibilidad.	Alta	13	HU-007

Tabla 6: Product backlog del sistema

Fuente: Elaboración propia basada en metodología SCRUM adaptada a plataformas low-code con IA (Atlassian, s.f.; Ideas2IT, 2025; Mendix, Mendix: Low-Code Application Development Platform, 2025; Scrum Institute, 2025).

Historias de usuario

ID	HU-001	Nombre	Recopilación de requisitos iniciales con métricas energéticas
Descripción		Como gerente de proyecto, quiero recopilar requisitos iniciales con integración de métricas energéticas para planificar aplicaciones ecoeficientes.	
Criterios de aceptación		1. Dado un conjunto de necesidades del cliente sin métricas ambientales, cuando se ingresen en la plataforma, entonces el sistema integrará automáticamente estimaciones de consumo energético inicial. 2. Dado un requisito incompleto, cuando se valide, entonces el sistema avisará de campos requeridos para incluir aspectos ecoeficientes. 3. Dado un conjunto de requisitos validados, cuando se archiven, entonces se generará un reporte preliminar de impacto ambiental. 4. Dado un requisito con métricas energéticas existentes, cuando se actualice, entonces el sistema recalculará las proyecciones de huella de carbono.	
Prioridad	Alta	Puntos de historia estimados	13

Tabla 7: Recopilación de requisitos iniciales con métricas energéticas.

Fuente: Elaboración propia.

ID	HU-002	Nombre	Diseño de arquitectura low-code con soporte IA
----	--------	--------	--

Descripción		Como arquitecto de software, quiero diseñar la arquitectura low-code con soporte IA para seleccionar componentes reutilizables que minimicen el impacto ambiental.	
Criterios de aceptación		1. Dado un conjunto de requisitos aprobados, cuando se inicie el diseño, entonces el IA sugerirá componentes low-code con bajo consumo energético. 2. Dado un componente no reutilizable, cuando se evalúe, entonces el sistema alertará sobre alternativas ecoeficientes disponibles. 3. Dado un diseño preliminar, cuando se valide, entonces se calculará el impacto ambiental proyectado. 4. Dado un cambio en requisitos, cuando se actualice la arquitectura, entonces el IA reoptimizará la selección de componentes automáticamente.	
Prioridad	Alta	Puntos de historia estimados	8

Tabla 8: Diseño de arquitectura low-code con soporte IA.

Fuente: Elaboración propia.

ID	HU-003	Nombre	Generación de código optimizado mediante IA
Descripción		Como desarrollador, quiero generar código optimizado automáticamente mediante IA para reducir ciclos redundantes y consumo de CPU en aplicaciones web/móviles.	
Criterios de aceptación		1. Dado un diseño de arquitectura aprobado, cuando se genere código, entonces el IA eliminará ciclos redundantes automáticamente.	

		2. Dado código con ineficiencias detectadas, cuando se procese, entonces se reducirá el consumo de CPU estimado en al menos un 70%. 3. Dado un lenguaje de programación seleccionado, cuando se compile, entonces el sistema validará la eficiencia energética. 4. Dado código generado, cuando se revise, entonces se proporcionarán sugerencias IA para mejoras adicionales en huella de carbono.	
Prioridad	Media	Puntos de historia estimados	13

Tabla 9: Generación de código optimizado mediante IA

Fuente: Elaboración propia.

ID	HU-004	Nombre	Ejecución de pruebas de funcionalidad con evaluación energética
Descripción		Como analista de QA, quiero ejecutar pruebas de funcionalidad con evaluación de eficiencia energética para validar el rendimiento eco-eficiente.	
Criterios de aceptación		1. Dado código generado, cuando se ejecuten pruebas funcionales, entonces se midan métricas de consumo energético en tiempo real. 2. Dado un fallo en funcionalidad, cuando se detecte, entonces se correlacione con impactos ambientales excesivos. 3. Dado pruebas completadas, cuando se reporte, entonces se confirme una reducción en emisiones de CO2 comparada con métodos tradicionales. 4. Dado un entorno de prueba, cuando se simule, entonces el sistema alertará sobre picos de ineficiencia energética.	

Trabajo Final de Grado – Prototipado Tecnológico

Prioridad	Media	Puntos de historia estimados	5
-----------	-------	------------------------------	---

Tabla 10: Ejecución de pruebas de funcionalidad con evaluación energética.

Fuente: Elaboración propia.

ID	HU-005	Nombre	Monitoreo de métricas de rendimiento con dashboard IA
Descripción		Como ingeniero de operaciones, quiero monitorear métricas de rendimiento en tiempo real con dashboard IA para detectar picos de consumo energético.	
Criterios de aceptación		1. Dado pruebas validadas, cuando se monitoree el tiempo de ejecución, entonces el dashboard IA visualice métricas energéticas en tiempo real. 2. Dado un pico de consumo detectado, cuando se analice, entonces se genere una alerta automática para optimización. 3. Dado datos históricos, cuando se consulte, entonces se compare con evaluaciones de rendimiento ecoeficientes. 4. Dado un dashboard actualizado, cuando se exporte, entonces se incluyan estimaciones de reducción de CO2.	
Prioridad	Baja	Puntos de historia estimados	8

Tabla 11: Monitoreo de métricas de rendimiento con dashboard IA.

Fuente: Elaboración propia.

ID	HU-006	Nombre	Refactorización de código con sugerencias IA
Descripción		Como desarrollador senior, quiero refactorizar código manualmente con sugerencias IA para optimizar huella de carbono post-generación.	

Criterios de aceptación		1. Dado código generado, cuando se inicie refactorización, entonces el IA proporcione sugerencias para reducir huella de carbono. 2. Dado una sugerencia aplicada, cuando se valide, entonces se mida la mejora en eficiencia energética. 3. Dado código obsoleto detectado, cuando se procese, entonces se eliminen estructuras ineficientes automáticamente. 4. Dado refactorización completada, cuando se archive, entonces se registre el impacto ambiental mejorado.	
Prioridad	Baja	Puntos de historia estimados	5

Tabla 12: Refactorización de código con sugerencias IA.

Fuente: Elaboración propia.

ID	HU-007	Nombre	Generación de reportes ambientales automáticos
Descripción		Como analista de sostenibilidad, quiero generar reportes ambientales automáticos con estimaciones de CO2 para comparar con métodos tradicionales.	
Criterios de aceptación		1. Dado métricas monitoreadas, cuando se solicite un reporte, entonces se genere automáticamente con estimaciones de CO2. 2. Dado datos comparativos, cuando se analicen, entonces se destaque la reducción del 70% en emisiones. 3. Dado un reporte incompleto, cuando se valide, entonces se avise de datos requeridos para precisión ambiental.	

		4. Dado un reporte final, cuando se exporte, entonces se incluya visualizaciones de impacto sostenible.	
Prioridad	Media	Puntos de historia estimados	8

Tabla 13: Generación de reportes ambientales automáticos.

Fuente: Elaboración propia.

ID	HU-008	Nombre	Despliegue inicial con revisión de métricas ecoeficientes
Descripción		Como DevOps especialista, quiero desplegar la aplicación inicial con revisión de métricas ecoeficientes para asegurar alineación con objetivos de sostenibilidad.	
Criterios de aceptación		1. Dado reportes ambientales aprobados, cuando se inicie despliegue, entonces se revise métricas ecoeficientes en el entorno de nube. 2. Dado un issue post-despliegue, cuando se detecte, entonces se correlacione con ineficiencias energéticas. 3. Dado despliegue completado, cuando se valide, entonces se confirme alineación con reducción de huella de carbono. 4. Dado feedback inicial, cuando se registre, entonces se optimice para futuras iteraciones sostenibles.	
Prioridad	Alta	Puntos de historia estimados	13

Tabla 14: Despliegue inicial con revisión de métricas ecoeficientes

Fuente: Elaboración propia.

Sprint backlog

El sprint backlog se elaboró para el primer sprint del prototipo, seleccionando historias prioritarias del product backlog (HU-001, HU-002 y HU-008) que impactan en el core del proyecto relacionado con la integración de IA en plataformas low-code para

desarrollo eco-eficiente. Cada historia incorpora tareas específicas y detalladas, con prioridades, estimados en días y estados, facilitando una planificación táctica para rastrear el progreso en la metodología SCRUM.

Sprint	Historia de usuario	ID	Tareas	Prioridad	Estimado	Estado
HU-001	Recopilación de requisitos iniciales con métricas energéticas	01	Definir interfaz para ingreso de requisitos con métricas ambientales.	Alta	2 días	Hecho
		02	Integrar API de estimación energética inicial en la plataforma low-code.	Media	3 días	Hecho
		03	Validar campos requeridos para aspectos ecoeficientes en el formulario.	Alta	2 días	En progreso
		04	Generar reporte preliminar de impacto ambiental	Media	4 días	Por hacer

Trabajo Final de Grado – Prototipado Tecnológico

			post- validación.			
		05	Recalcular proyecciones de huella de carbono al actualizar requisitos.	Alta	2 días	Por hacer
HU-002	Diseño de arquitectura low-code con soporte IA	01	Sugerir componentes low-code con bajo consumo energético vía IA.	Alta	2 días	Hecho
		02	Evaluar alternativas ecoficientes para componentes no reutilizables.	Media	2 días	Hecho
		03	Calcular impacto ambiental proyectado en el diseño preliminar.	Alta	1 día	En progreso
		04	Reoptimizar selección de	Media	3 días	Por hacer

			componentes al cambiar requisitos.			
HU-008	Despliegue inicial con revisión de métricas ecoeficientes	01	Revisar métricas ecoeficientes en el entorno de nube durante despliegue.	Alta	3 días	Hecho
		02	Correlacionar issues post-despliegue con ineficiencias energéticas.	Media	4 días	En progreso
		03	Validar alineación con reducción de huella de carbono al completar despliegue.	Alta	2 días	Por hacer
		04	Registrar feedback inicial para optimizaciones sostenibles futuras.	Media	4 días	Por hacer

Tabla 15: Sprint Backlog HU-001-HU-002-HU-008

Fuente: Elaboración propia.

Estructura de datos

La estructura de datos del sistema se modeló mediante un Diagrama Entidad-Relación (DER), con el fin de representar las entidades principales, sus atributos, métodos y relaciones necesarias para soportar la plataforma low-code integrada con IA enfocada en aplicaciones ecoeficientes.

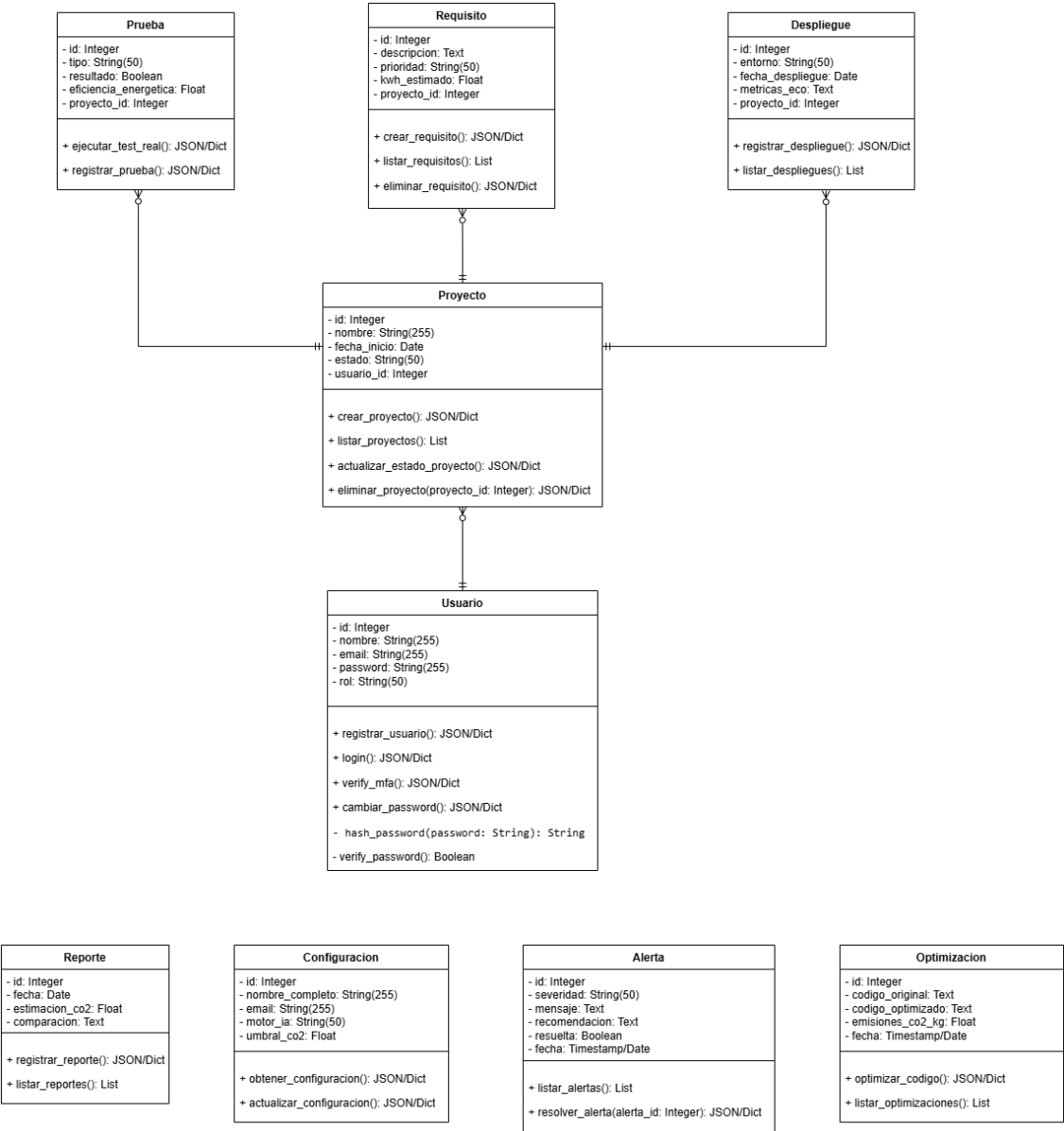


Figura 4 - Diagrama de Entidad-Relación de Plataforma Low-Code

Fuente: Elaboración propia en Lucidchart.

El diagrama expuesto representa el modelo estructural y de comportamiento de la plataforma EcoDev, integrando el diseño relacional de la base de datos con la encapsulación propia de una arquitectura orientada a servicios (API RESTful). El sistema

se divide lógicamente en dos grandes áreas: un núcleo transaccional con estricta integridad referencial y un módulo de telemetría global orientado a los principios de Green IT.

En la especificación de los métodos del diagrama, se establece JSON/Dict como el tipo de dato de retorno. Esto responde directamente a la naturaleza de la API construida sobre FastAPI. Al optar por un modelo de dominio rico que prescinde de clases controladoras tradicionales, son las propias entidades o servicios del dominio los que encapsulan la lógica de negocio y procesan las peticiones. El resultado de estas operaciones se devuelve en forma de diccionarios nativos, los cuales el entorno de trabajo serializa automáticamente a formato JSON.

El eje central del flujo de trabajo recae sobre la entidad *Proyecto*, la cual actúa como el nodo aglutinador de todo el ciclo de vida del software sostenible. Las relaciones de cardinalidad se estructuran de la siguiente manera:

Usuario – Proyecto: Existe una relación de uno a muchos (1:N). La entidad Usuario gestiona el control de acceso estructurado mediante roles y credenciales encriptadas. Un usuario autenticado es propietario de múltiples proyectos, garantizando la trazabilidad de quién inicializa cada medición energética.

Proyecto - Requisito: Cardinalidad 1:N. Cada proyecto está compuesto por múltiples historias de usuario o Requisitos. Esta relación permite segmentar y calcular el impacto en kWh estimado fase por fase, derivando de un proyecto unificado.

Proyecto - Prueba y Proyecto - Despliegue: Ambas mantienen una relación 1:N con la entidad central. Las pruebas de ejecución física (que validan la sintaxis y el consumo real) y los despliegues en la nube (CI/CD) quedan registrados como un historial inmutable anclado al Proyecto original. Si un proyecto es eliminado del sistema, la integridad referencial garantiza la eliminación en cascada de sus requisitos, pruebas y despliegues asociados, liberando eficientemente el espacio de almacenamiento.

A diferencia del núcleo transaccional, el sistema implementa un conjunto de entidades globales (*Optimizacion, Alerta, Reporte y Configuracion*) que operan de manera transversal a todos los proyectos.

Optimizacion y Reporte: Actúan como un registro de auditoría ambiental (logs físicos). Almacenan los metadatos de las refactorizaciones realizadas por Inteligencia

Artificial y los porcentajes de ahorro de CO2 en comparación con el desarrollo tradicional.

Alerta: Se retroalimenta del tracker de emisiones en tiempo real. Constituye una entidad independiente que almacena los picos anómalos de consumo de hardware durante la generación de código, facilitando el monitoreo continuo sin acoplarse a un requerimiento específico.

Configuración: Entidad de instancia única que dicta las reglas de negocio globales, tales como el umbral máximo de dióxido de carbono permitido antes de bloquear un despliegue en producción.

Prototipos de interfaces de pantallas

Prototipo 1: Panel de Operaciones y Telemetría Sostenible

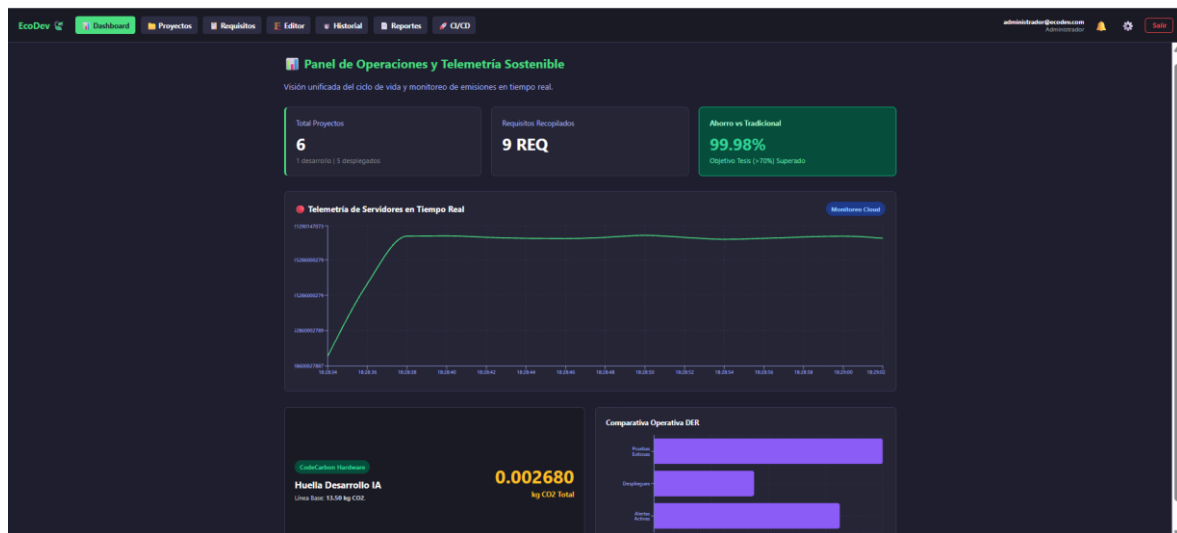


Figura 5 - Panel de Operaciones y Telemetría Sostenible

Fuente: Elaboración propia en Figma.

El Panel de Operaciones y Telemetría Sostenible actúa como el núcleo analítico de la plataforma. Proporciona una visión unificada del ciclo de vida del software, destacando en tiempo real las métricas de consumo de los servidores y la huella de carbono generada por el desarrollo con Inteligencia Artificial (CodeCarbon). Además, cuantifica el porcentaje de ahorro energético frente a metodologías de codificación tradicionales, validando visualmente el cumplimiento del objetivo de ecoeficiencia estipulado para el sistema.

Prototipo 2: Pantalla de Gestión de Proyectos

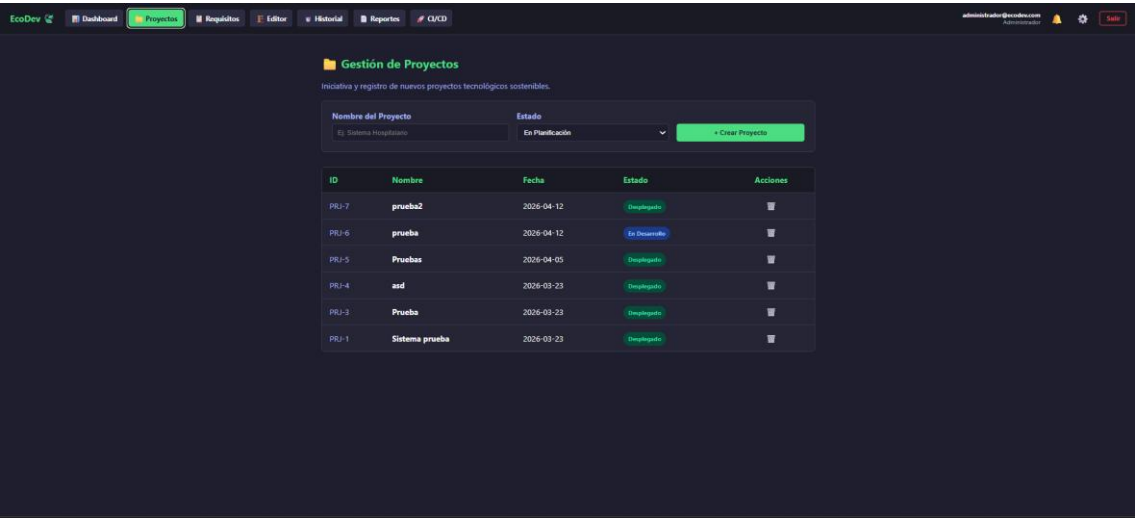


Figura 6 - Pantalla de Gestión de Proyectos

Fuente: Elaboración propia en Figma.

Esta interfaz centraliza el registro y seguimiento de las iniciativas tecnológicas. Permite estructurar nuevos proyectos definiendo su estado de madurez operativa (planificación, desarrollo o despliegue) y asociándolos a identificadores únicos. Su diseño facilita la administración del portafolio, garantizando que cada pieza de software mantenga una trazabilidad ambiental rigurosa desde su fase inicial de concepción.

Prototipo 3: Pantalla de Recopilación de Requisitos (Green IT)

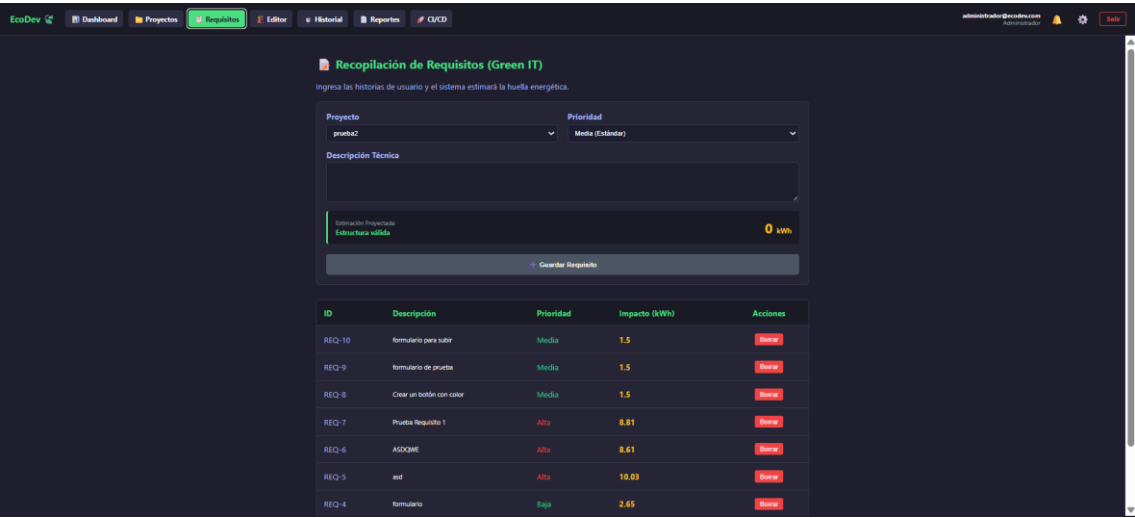


Figura 7 - Pantalla de Recopilación de Requisitos (Green IT)

Fuente: Elaboración propia en Figma.

Módulo diseñado para la captura de historias de usuario bajo el paradigma Green IT. Al ingresar la descripción técnica y la prioridad del requerimiento, el sistema procesa la información y realiza una estimación proyectada del impacto energético en kilovatios-hora (kWh). Si la estructura carece de suficiencia técnica, el sistema emite una alerta, asegurando que la fase de planificación incorpore la sostenibilidad como un requisito no funcional ineludible.

Prototipo 4: Pantalla de Editor y Arquitectura Low-Code

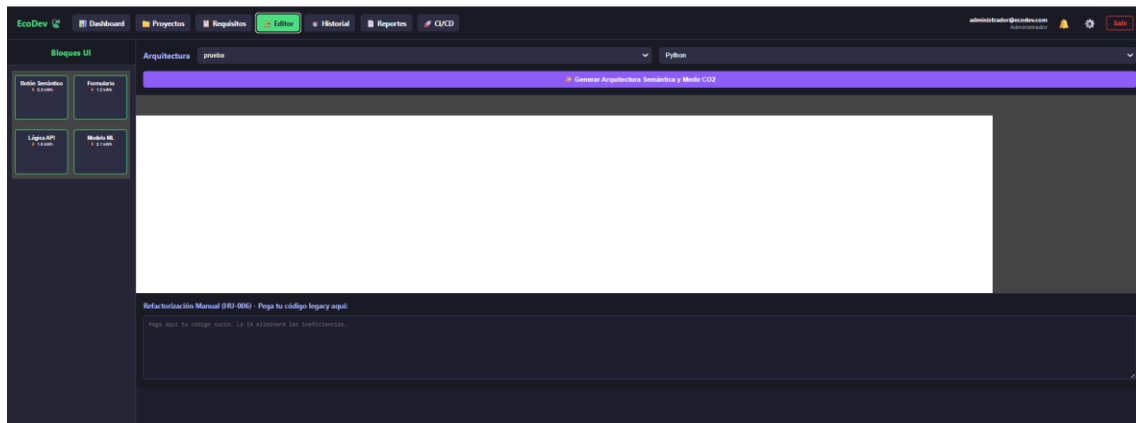


Figura 8 - Pantalla de Editor y Arquitectura Low-Code

Fuente: Elaboración propia en Figma.

Entorno de desarrollo visual que integra una biblioteca de componentes semánticos pre-evaluados por su consumo energético (kWh). El lienzo de trabajo permite estructurar la interfaz de forma ágil, mientras que un panel inferior habilita la carga del código heredado para su refactorización manual. Desde aquí se invoca al motor de IA para generar arquitecturas limpias y medir el costo en CO2 de dicho proceso de optimización.

Prototipo 5: Pantalla de Historial de Optimizaciones con IA

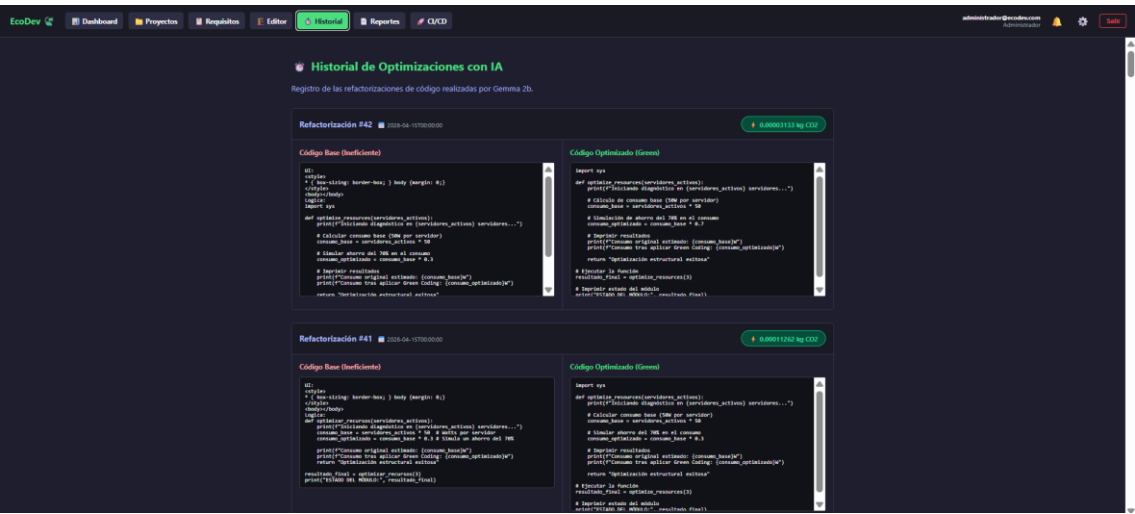


Figura 9 - Pantalla de Historial de Optimizaciones con IA

Fuente: Elaboración propia en Figma.

Pantalla de auditoría técnica que documenta cada refactorización ejecutada por el modelo LLM. Expone una comparativa directa entre el código base ineficiente y el código resultante optimizado (Green Code). Asimismo, registra de forma inmutable las emisiones exactas de CO2 que el hardware consumió durante la inferencia del modelo, garantizando transparencia absoluta sobre el gasto energético de la propia inteligencia artificial.

Prototipo 6: Pantalla de Generador de Reportes Ambientales

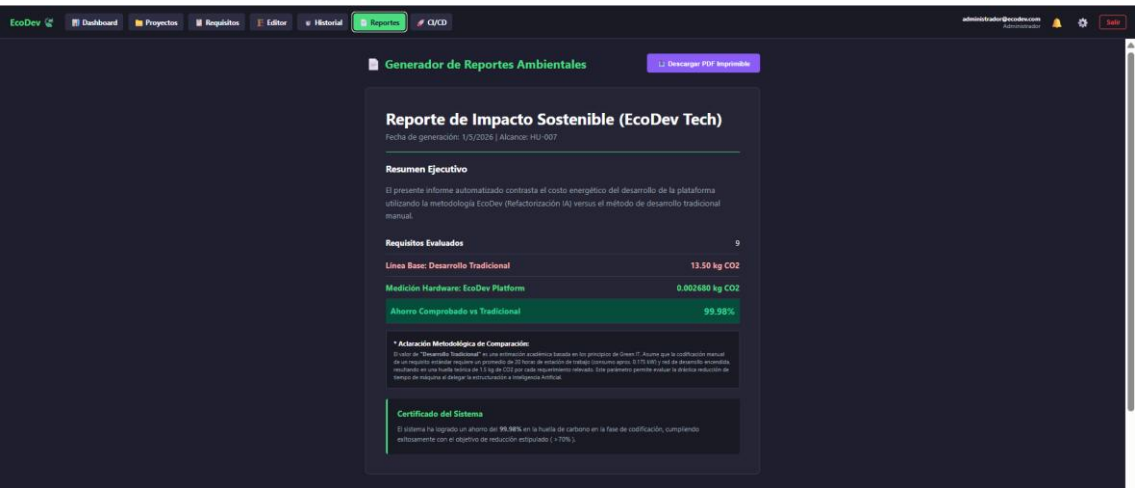


Figura 10 - Pantalla de Generador de Reportes Ambientales

Fuente: Elaboración propia en Figma.

Herramienta de síntesis que contrasta el costo energético del desarrollo automatizado frente al método tradicional. Permite la generación de documentos exportables en formato PDF que detallan los requisitos evaluados, la línea base teórica y el porcentaje de ahorro comprobado. Incluye un historial de emisiones que funciona como un certificado formal de validación de los objetivos de reducción energética del sistema.

Prototipo 7: Pantalla de Pipeline de CI/CD (Validación y Despliegue)

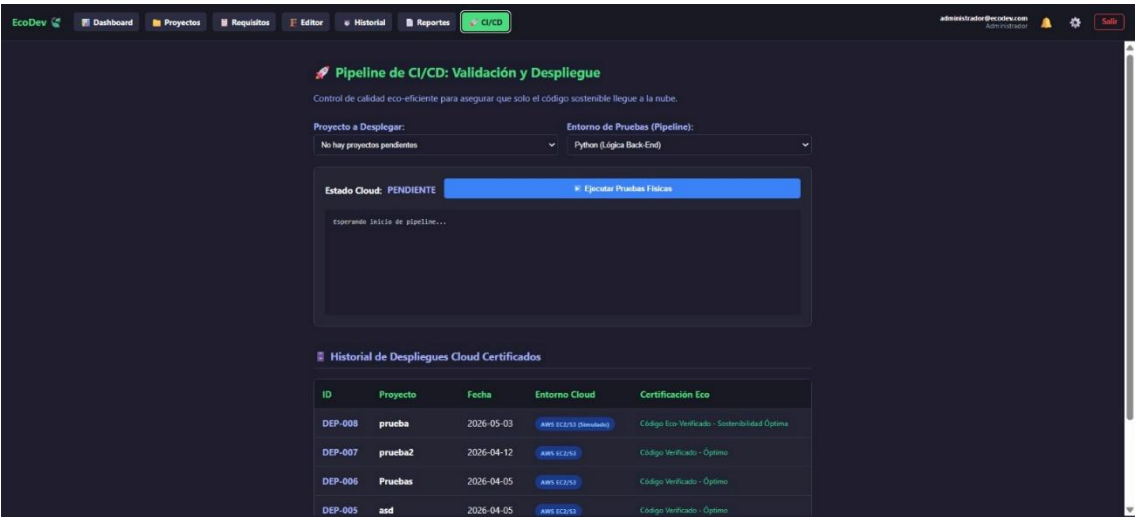


Figura 11 - Pantalla de Pipeline de CI/CD (Validación y Despliegue)

Fuente: Elaboración propia en Figma.

Centro de control de calidad eco-eficiente que actúa como barrera preventiva para el despliegue de software no sustentable. Ejecuta pruebas físicas del código en un entorno simulado y evalúa si la huella energética excede el umbral permitido. En caso de superar dicho límite, el sistema bloquea automáticamente el pase a producción (entornos de nube), exigiendo una nueva refactorización para certificar su sostenibilidad.

Prototipo 8: Centro de Alertas Ambientales

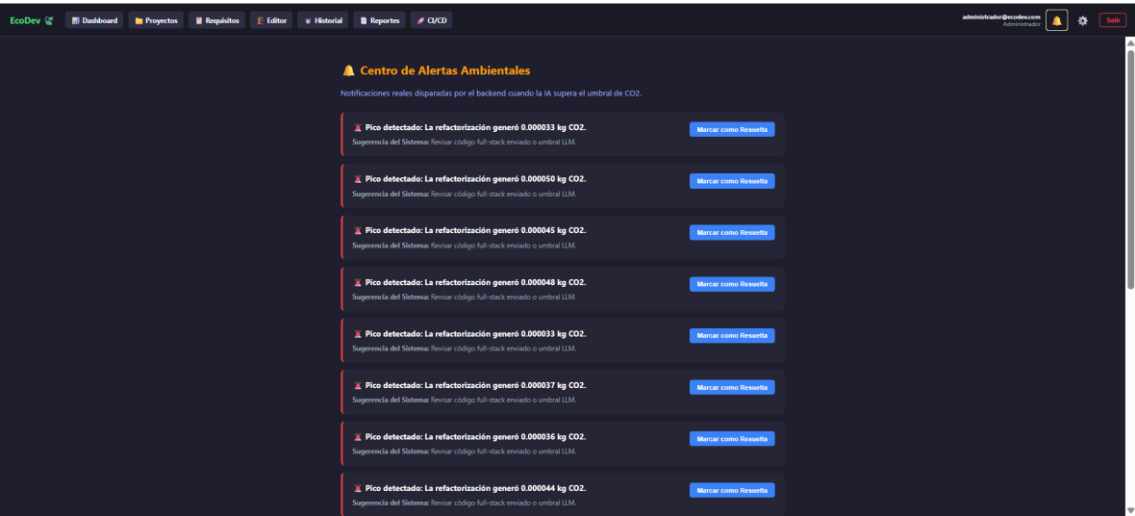


Figura 12 - Pantalla de Centro de Alertas Ambientales

Fuente: Elaboración propia en Figma.

Sistema de notificaciones dinámicas disparadas por el *backend* cuando los procesos superan los umbrales de emisión configurados. Cada alerta clasifica la severidad del evento, detalla el exceso de CO2 generado en la refactorización y ofrece sugerencias sistémicas orientadas a la mitigación. Los administradores pueden gestionar, auditar y marcar estas incidencias operativas como resueltas una vez aplicado el ajuste pertinente.

Prototipo 9: Pantalla de Configuración de Plataforma y Seguridad

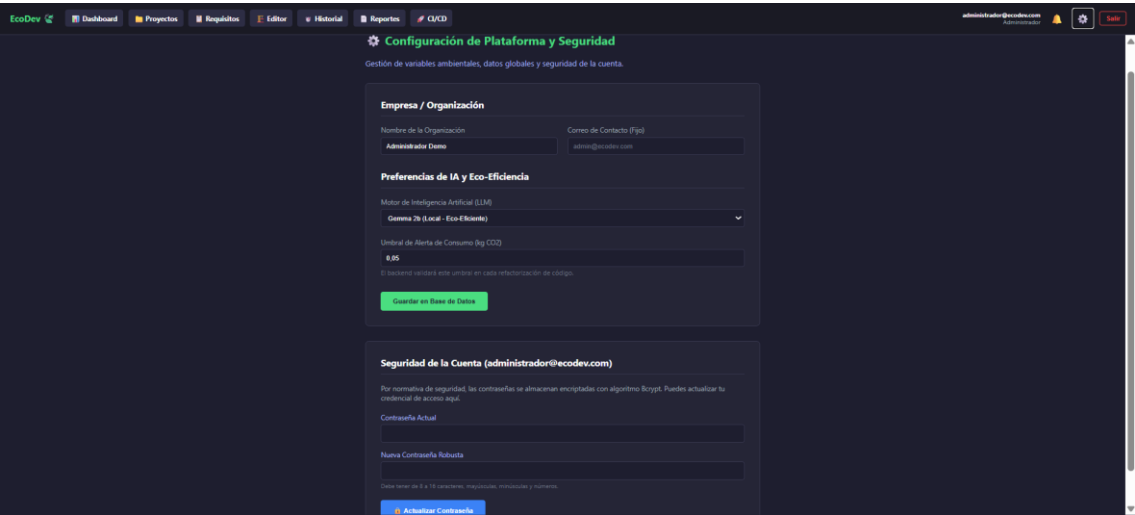


Figura 13 - Pantalla de Configuración de Plataforma y Seguridad

Fuente: Elaboración propia en Figma.

Módulo de administración donde se definen las variables críticas del entorno global. Permite seleccionar el motor de Inteligencia Artificial activo y establecer el límite estricto de tolerancia de emisiones (kg CO₂) que regulará todo el pipeline de CI/CD. Adicionalmente, gestiona la seguridad del usuario, garantizando el almacenamiento cifrado de credenciales mediante el algoritmo Bcrypt.

Prototipo 10: Pantalla de Autenticación y Acceso Seguro

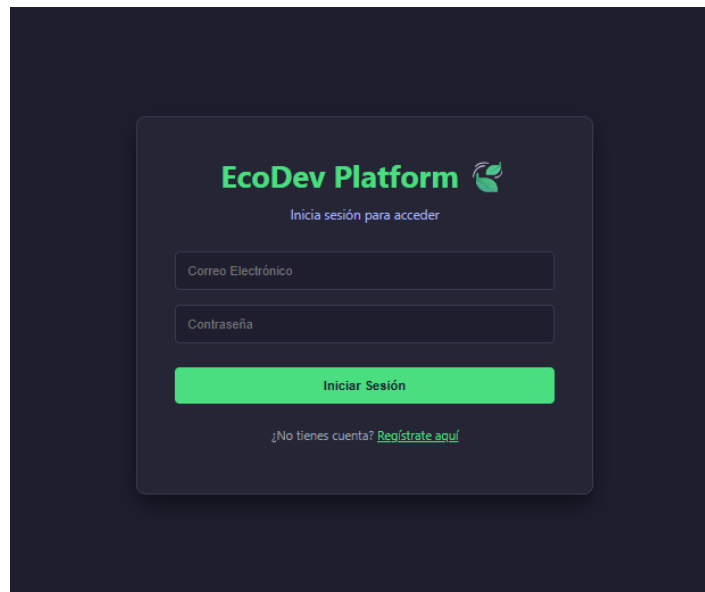


Figura 14 - Pantalla de Autenticación y Acceso Seguro

Fuente: Elaboración propia en Figma.

Punto de entrada a la plataforma que asegura la protección de las métricas ambientales y la infraestructura subyacente. Implementa un sistema de validación de contraseñas robustas y Autenticación Multifactor (MFA). Su diseño de seguridad integral aplica un estricto control basado en roles, garantizando que el menú de navegación oculte dinámicamente los módulos no autorizados para el usuario, protegiendo así la integridad de la operación.

Encadenamiento Lógico de Prototipos de Interfaces

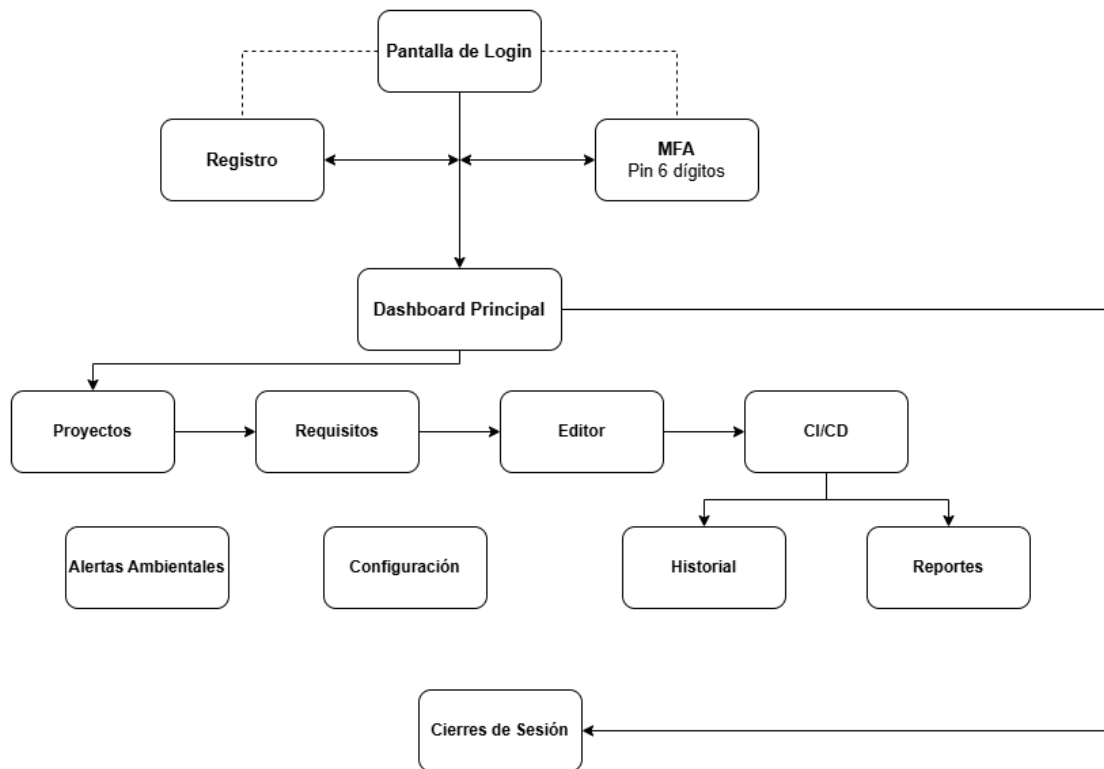


Figura 15 - Encadenamiento Lógico de Prototipos de Interfaces

Fuente: Elaboración propia en Draw.io.

El presente diagrama ilustra el flujo de navegabilidad y la arquitectura de interacción de la plataforma EcoDev. A diferencia de los modelos de navegación lineales tradicionales, la interfaz está estructurada como una Single Page Application (SPA) provista de un enrutamiento dinámico y un sistema de control de acceso basado en roles (RBAC). Esta arquitectura permite que la experiencia de usuario se articule en torno a un eje central, garantizando una navegación fluida y contextual. El modelo de encadenamiento se divide en las siguientes áreas funcionales:

Bloque de Acceso y Autenticación: El flujo inicia en un entorno seguro gestionado por estados internos, donde convergen el inicio de sesión, el registro de nuevos usuarios y la Autenticación Multifactor (MFA). Una vez validadas las credenciales, el sistema redirige al usuario hacia el núcleo operativo.

Hub Central y Navegación Transversal: El Dashboard Principal actúa como la pantalla de aterrizaje, ofreciendo una vista unificada de la telemetría del sistema. De este

componente se desprende una barra de navegación global y persistente, la cual renderiza dinámicamente sus opciones dependiendo de los permisos y el rol del usuario autenticado.

Flujo Secuencial de Negocio: Representa el ciclo de vida del desarrollo de software sostenible. El encadenamiento lógico guía al usuario a través de un proceso altamente cohesivo:

1. Inicia con la creación y gestión en el módulo de Proyectos.
2. Avanza hacia la recopilación y estimación energética en Requisitos.
3. Transita al Editor Low-Code, un entorno unificado que consolida el diseño de la arquitectura visual, la refactorización de código y la optimización generativa impulsada por IA.
4. Culmina en el pipeline de CI/CD, donde convergen las pruebas físicas del código y las validaciones de huella de carbono previas al despliegue en entornos de nube.

Módulos de Auditoría y Soporte: Elementos como el Historial de Optimizaciones, el Generador de Reportes, el Centro de Alertas Ambientales y la Configuración de Plataforma operan como módulos transversales. El usuario puede acceder a ellos en cualquier momento desde la barra de navegación sin interrumpir el flujo secuencial de desarrollo, permitiendo una monitorización continua y una gestión integral del impacto ambiental de la plataforma.

Diagrama de arquitectura

El diagrama de arquitectura se presenta como una representación gráfica de la estructura general del prototipo, ilustrando los componentes principales y sus interrelaciones para facilitar la creación sostenible de aplicaciones ecoeficientes mediante una plataforma low-code integrada con IA. Se organiza en capas lógicas: la capa de presentación (frontend) utiliza interfaces drag-and-drop basadas en JavaScript para permitir la interacción intuitiva del usuario con requisitos y diseños; la capa de lógica de negocio (backend) integra modelos de IA en Python para optimización y refactorización automática de código, conectada a herramientas de medición como CodeCarbon y Climatiq para estimar emisiones de CO2 en tiempo real; la capa de datos emplea bases de datos como MySQL para almacenar métricas ambientales y historiales de optimizaciones; y la capa de infraestructura se soporta en entornos de nube como AWS, optimizados para bajo consumo energético. Las flechas indican flujos bidireccionales de datos, asegurando

que las sugerencias IA se retroalimenten con métricas ambientales para reducir ineficiencias, alineado con el diagnóstico de problemas en procesos de desarrollo y el alcance del prototipo que demuestra procesos clave sin implementación completa. Esta arquitectura holística minimiza recursos computacionales, promoviendo despliegues escalables y sostenibles, como se detalla en la metodología SCRUM adoptada.

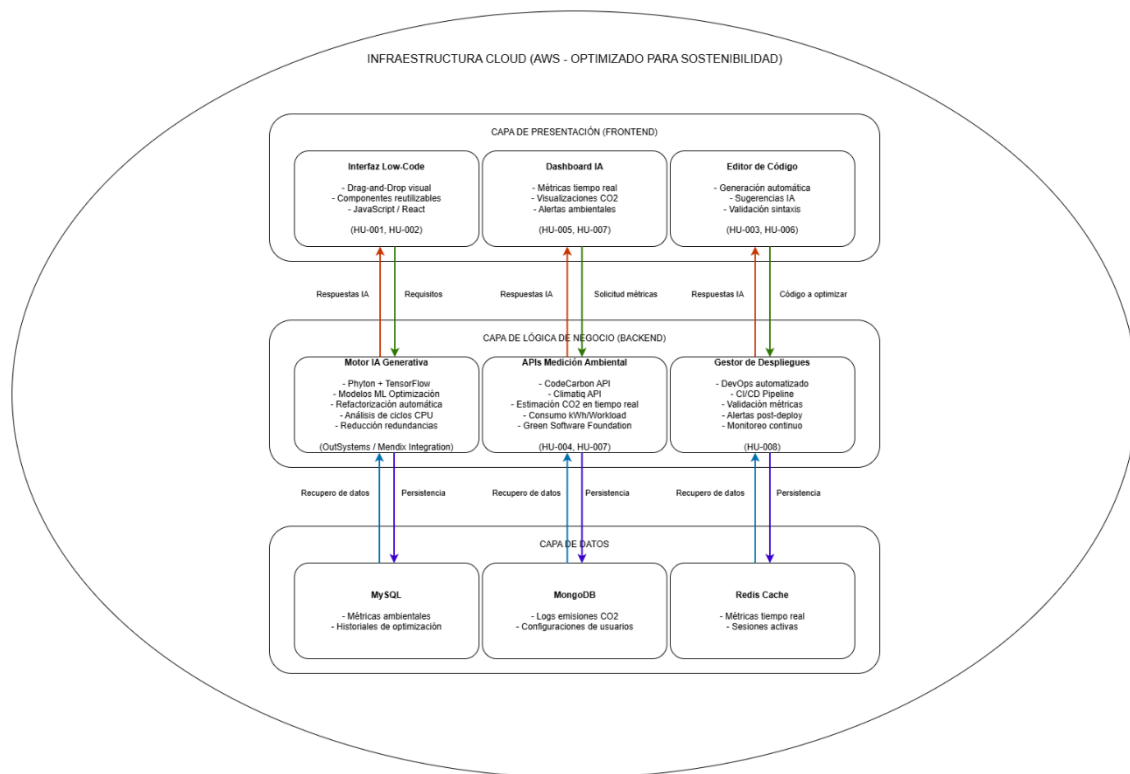


Figura 16 - Diagrama de arquitectura de Plataforma Low-Code con IA

Fuente: Elaboración propia en Draw.io.

Seguridad

En el marco de este prototipo tecnológico, se establecieron medidas de seguridad para proteger el acceso a la plataforma y la integridad de los datos que esta genera (métricas de ecoeficiencia, configuraciones de IA y código optimizado).

El acceso al prototipo se gestiona mediante un sistema de autenticación robusto. La política de credenciales requiere un correo electrónico de usuario (como identificador único) y una contraseña que debe cumplir con estándares de complejidad (longitud de 8 a 16 caracteres, uso de mayúsculas, minúsculas y números) para prevenir ataques de fuerza bruta.

Para mitigar riesgos de intrusión y suplantación de identidad, se justifica la implementación de la Autenticación Multifactor (MFA, definido en el Marco Teórico Referencial). Para este prototipo, la MFA se integra con el módulo de IA de la plataforma para analizar patrones de comportamiento y detectar accesos anómalos, añadiendo una capa de seguridad proactiva alineada con las capacidades del sistema.

Respecto a la política de respaldo de información, se definió una política integral para el resguardo y la disponibilidad de los datos del prototipo, priorizando enfoques ecoeficientes. Para el almacenamiento de los datos críticos generados (métricas de IA, logs de optimización y configuraciones de proyectos low-code), se seleccionó el servicio AWS S3. Esta elección se justifica por su alta durabilidad y su capacidad nativa para integrarse con cifrado AES-256, tecnologías descritas en el Marco Teórico. Esta combinación asegura la confidencialidad de la información sensible en reposo y en tránsito. La periodicidad del respaldo se justifica por la naturaleza de los datos, se programaron respaldos diarios completos y respaldos incrementales cada hora. Esta frecuencia se considera necesaria para garantizar un Punto de Recuperación (RPO) menor a una hora, vital para los datos generados por la IA. El acceso a dichos respaldos se restringe estrictamente mediante roles de AWS IAM (ver Marco Teórico Referencial), aplicando el principio de mínimo privilegio solo al personal de administración de sistemas.

Dado que el prototipo está diseñado para el monitoreo en tiempo real de métricas ambientales y la asistencia constante de la IA, se requiere una disponibilidad 24/7. Para cumplir este requisito, se justifica el uso de una arquitectura de Alta Disponibilidad (HA), fundamentada en los conceptos teóricos de clústeres redundantes y balanceo de carga en AWS. Asimismo, se emplea la geo-replicación de los respaldos para asegurar la continuidad operativa y la recuperación ante desastres regionales.

Análisis de costos

En el marco del proyecto se realiza un análisis de costos que abarca los recursos humanos para el desarrollo y mantenimiento, así como los recursos de hardware y software necesarios. Este análisis se basa en la metodología SCRUM adoptada, considerando un equipo multidisciplinario para el desarrollo durante un período estimado de entre 1 y 3 meses y un mantenimiento trimestral post-implementación. Los costos se

expresan en pesos argentinos (ARS), utilizando la tasa de conversión del Banco de la Nación Argentina (BNA) para valores en dólares estadounidenses (USD).

Costos de personal para el desarrollo

Los costos de personal para el desarrollo del proyecto se detallan en la siguiente tabla, considerando roles clave en SCRUM alineados con el Consejo Profesional de Ciencias Informáticas de Buenos Aires (CPCIBA), como Administrador Profesional de Proyectos Senior (para Scrum Master y PM), Analista de Negocios (para Product Owner), Analista Programador Senior (para desarrolladores low-code), Consultor BI/IA (para especialistas en IA), Consultor Infraestructura (para DevOps/cloud), Analista de Usabilidad/UX (para diseño de interfaces), Analista de Calidad Senior (para QA/testing) y Administrador de Seguridad Informática Senior (para seguridad). Se asume un equipo de 9 personas trabajando a tiempo completo durante 3 meses, con honorarios mensuales ajustados a valores de 2025.

Rol	Cantidad de informáticos	Honorario mensual	Cantidad de meses	Costo total para desarrollo
Analista de Negocios (Product Owner)	1	\$1.484.420	3	\$ 4.453.260
Administrador Profesional de Proyectos Senior (Scrum Master/PM)	1	\$2.125.200	3	\$6.375.600
Analista Programador Senior (Desarrollador Low-Code)	1	\$2.144.520	3	\$6.433.560
Consultor BI/IA (Especialista en IA)	1	\$2.524.480	2	\$5.048.960

Consultor Infraestructura (Ingeniero DevOps)	1	\$2.205.700	2	\$4.411.400
Analista de Usabilidad/UX (Diseñador UX)	1	\$1.638.980	1	\$1.638.980
Analista de Calidad Senior (QA/Tester)	1	\$1.477.980	2	\$2.955.960
Administrador de Seguridad Informática Senior	1	\$1.838.620	2	\$3.677.240
Total	9	-	-	\$34.994.960

Tabla 16 - Costos del personal de desarrollo

Fuente: Elaboración propia (CPCIBA, 2025).

Costos de operación del personal

Para el mantenimiento del proyecto una vez implementado, se considera un equipo reducido según necesidades, con un enfoque trimestral en actualizaciones, monitoreo eco-eficiente y soporte. Se incluyen roles esenciales como Consultor Infraestructura (DevOps), Soporte Técnico (para soporte de usuario), Consultor BI/IA (para IA ocasional), Administrador de Seguridad Informática Senior (para seguridad continua) y Analista de Calidad Senior (para testing post-lanzamiento).

Rol	Cantidad	Honorario mensual	Meses	Costo total de operación
Consultor Infraestructura (Ingeniero DevOps)	1	\$2.205.700	3	\$6.617.100

Soporte Técnico	1	\$1.210.720	3	\$3.632.160
Consultor BI/IA	1	\$2.524.480	3	\$7.573.440
Administrador de Seguridad Informática Senior	1	\$1.838.620	3	\$5.515.860
Analista de Calidad Senior	1	\$1.477.980	3	\$4.433.940
Total Trimestre	5	-	-	\$27.772.500

Tabla 17 - Costo de operación del personal

Fuente: Elaboración propia (CPCIBA, 2025).

Costos de hardware y software

Los recursos de hardware y software incluyen entornos de trabajo low-code, modelos de IA, herramientas de medición de huella de carbono y entornos de nube. Los costos se obtienen de proveedores oficiales, considerando licencias o costos por uso para software, y adquisiciones únicas para hardware (como estaciones de trabajo para el equipo de desarrollo). Los valores en USD y Euros se convierten a pesos argentinos utilizando la tasa BNA de 1 USD = \$1.515 y 1 euro = \$1.780 como se muestra a continuación.

Cotización Billetes	Cotización Divisas	
24/10/2025	Compra	Venta
Dolar U.S.A	1465,00	1515,00
Euro	1680,00	1780,00
Real *	27200,00	28700,00
Ver histórico		
Hora Actualización: 17:04		
(*) cotización cada 100 unidades.		

Tabla 18 - Cotización dólar BNA

Fuente: Elaboración por BNA (BNA, 2025).

Recurso	Tipo	Cantidad/Duración	Total (ARS)
OutSystems (Enterprise License)	Software	3 meses	\$13.748.625
Mendix (Standard Plan)	Software	3 meses	\$4.806.000
Appsmith Enterprise	Software	3 meses	\$7.500
Microsoft Power Apps Premium	Software	3 meses	\$1.363.500

Cloud Carbon Footprint	Software	3 meses	\$0
CodeCarbon	Software	3 meses	\$0
Climatiq API (Pro Plan)	Software	3 meses	\$2.272.500
AWS S3 (Almacenamiento 100 GB/mes + Operaciones)	Software/Cloud	3 meses	\$159.075
Estaciones de Trabajo (Laptops Dell para Desarrollo)	Hardware	9 (una por persona)	\$13.635.000
Total	-	-	\$35.992.200

Tabla 19 - Costo de hardware y software

Fuente: Elaboración propia (OutSystems, 2025; Mendix, 2025; Appsmith, 2025; Microsoft, 2025; CodeCarbon, 2025; Climatiq, 2025; AWS, 2025; Cloudwards, 2025).

El costo total para realizar todo este proyecto, incluyendo desarrollo (\$34.994.960), operación (\$27.772.500) y recursos de hardware y software (\$35.992.200), asciende a \$98.759.660.

Análisis de riesgo

Para realizar un análisis estructurado, los riesgos identificados para el prototipo tecnológico se han clasificado en cinco categorías principales. A continuación, se presenta la definición de cada una de estas categorías, adaptadas al contexto de este proyecto:

- **Riesgos del Proyecto:** Se refieren a las incertidumbres que afectan directamente la gestión, el cronograma y el presupuesto del proyecto. Involucran la planificación, los recursos asignados, la dependencia de proveedores tecnológicos (como AWS o OutSystems) y el cumplimiento de los plazos definidos en la metodología SCRUM.
- **Riesgos Técnicos:** Son aquellos que surgen de la complejidad tecnológica del prototipo. Se centran en la correcta funcionalidad, el rendimiento y la integración de los componentes, como la compatibilidad entre los entornos de trabajo low-code, los modelos de inteligencia artificial y las APIs de medición ambiental (CodeCarbon, ClimaTiq).
- **Riesgos de Seguridad:** Involucran las amenazas a la confidencialidad, integridad y disponibilidad de la plataforma y los datos que esta procesa (código fuente, métricas de CO₂). Dada la arquitectura en la nube (AWS), esto incluye vulnerabilidades de datos, fugas de información y la gestión de accesos no autorizados.
- **Riesgos Ambientales:** Refieren a la posibilidad de que el prototipo no cumpla sus objetivos centrales de ecoeficiencia. Esto incluye el riesgo de que la propia plataforma (especialmente la IA) genere un consumo energético elevado o que las mediciones de huella de carbono resulten incorrectas.
- **Riesgos de Gestión:** Se centran en los desafíos operativos y de adopción por parte de los usuarios finales (desarrolladores, arquitectos de software). Incluyen la resistencia al cambio de los equipos de desarrollo tradicionales o la falta de alineación con las nuevas métricas ecoeficientes propuestas por la plataforma.

ID	Tipo	Riesgo	Causa
RG-01	Del proyecto	Retrasos en el cronograma de desarrollo	Dependencia de integraciones externas con proveedores como OutSystems y AWS, que podrían experimentar interrupciones en servicios o actualizaciones imprevistas.

RG-02	Del proyecto	Sobrepaso del presupuesto asignado	Fluctuaciones en costos de licencias de herramientas low-code e IA, agravadas por necesidades adicionales de optimización eco-eficiente no previstas inicialmente.
RG-03	Técnico	Fallos en la integración de IA con la plataforma low-code	Incompatibilidades entre modelos de IA generativa (como en Appsmith o Power Apps) y frameworks low-code, derivadas de actualizaciones asincrónicas en APIs.
RG-04	Técnico	Ineficiencia energética en aplicaciones generadas	Generación de código subóptimo por IA que no prioriza métricas de huella de carbono, contradiciendo los objetivos de sostenibilidad del proyecto.
RG-05	De seguridad	Vulnerabilidades de datos y fugas de información	Exposición de datos sensibles durante la integración low-code con IA, causada por configuraciones predeterminadas inadecuadas en entornos cloud (AWS).
RG-06	De seguridad	Ataques de suplantación de identidad o autorización indebida	Falta de robustez en mecanismos de autenticación multifactor (MFA) en plataformas low-code, permitiendo accesos no autorizados a componentes de IA.
RG-07	Ambiental	Aumento inesperado en la huella de carbono del sistema	Alto consumo computacional de modelos de IA durante el entrenamiento o inferencia, no mitigado adecuadamente por herramientas como CodeCarbon o Climatiq.
RG-08	De gestión	Falta de adopción por usuarios finales	Resistencia al cambio en entornos informáticos tradicionales, donde la transición a aplicaciones ecoeficientes low-code no se comunica efectivamente.
RG-09	Técnico	Errores en la medición de métricas ambientales	Imprecisiones en herramientas como Cloud Carbon debido a datos regionales desactualizados sobre fuentes de energía en Argentina.

RG-10	Del proyecto	Dependencia excesiva de proveedores externos	Interrupciones en servicios de nube (AWS) por factores geopolíticos o ambientales, afectando la disponibilidad 24/7.
-------	--------------	--	--

Tabla 20 - Clasificación de riesgos

Fuente: Elaboración propia (Legit Security, 2025; The AI Journal, 2025; Mtex Tech, 2025).

Se presenta la matriz de riesgos del proyecto, donde se evalúa cualitativamente la probabilidad de ocurrencia y el impacto potencial de cada riesgo identificado, utilizando una escala de alta, media y baja. Esta evaluación facilita la priorización de estrategias de mitigación.

ID	Tipo	Riesgo	Probabilidad	Impacto
RG-01	Del proyecto	Retrasos en el cronograma de desarrollo	Media	Alta
RG-02	Del proyecto	Sobrepaso del presupuesto asignado	Media	Media
RG-03	Técnico	Fallos en la integración de IA con la plataforma low-code	Alta	Alta
RG-04	Técnico	Ineficiencia energética en aplicaciones generadas	Media	Media
RG-05	De seguridad	Vulnerabilidades de datos y fugas de información	Alta	Alta
RG-06	De seguridad	Ataques de suplantación de identidad o autorización indebida	Media	Alta
RG-07	Ambiental	Aumento inesperado en la huella de	Baja	Media

		carbono del sistema		
RG-08	De gestión	Falta de adopción por usuarios finales	Media	Baja
RG-09	Técnico	Errores en la medición de métricas ambientales	Baja	Baja
RG-10	Del proyecto	Dependencia excesiva de proveedores externos	Alta	Media

Tabla 21 - matriz de riesgos del proyecto.

Fuente: Elaboración propia.

Se realiza un análisis cuantitativo de los riesgos identificados en el proyecto, asignando valores numéricos a la probabilidad de ocurrencia y al impacto, basados en la matriz de riesgos. Esta cuantificación permite priorizar los riesgos para el desarrollo de planes de contingencia, considerando el enfoque en plataformas low-code con IA y aspectos de sostenibilidad.

				Impacto				
				Muy bajo	Bajo	Medio	Alto	Muy alto
				1	2	3	4	5
Probabilidad	Muy alta	90%	0.9	0.9	1.8	2.7	3.6	4.5
	Alta	70%	0.7	0.7	1.4	2.1	2.8	3.5
	Media	50%	0.5	0.5	1	1.5	2	2.5
	Baja	30%	0.3	0.3	0.6	0.9	1.2	1.5
	Muy baja	10%	0.1	0.1	0.2	0.3	0.4	0.5

Tabla 22 - Matriz de riesgo

Fuente: Elaboración propia.

ID	Riesgo	Probabilidad de Ocurrencia	Impacto
RG-01	Retrasos en el cronograma de desarrollo	0.55	4
RG-02	Sobrepaso del presupuesto asignado	0.45	3
RG-03	Fallos en la integración de IA con la plataforma low-code	0.75	4
RG-04	Ineficiencia energética en aplicaciones generadas	0.50	3
RG-05	Vulnerabilidades de datos y fugas de información	0.80	4
RG-06	Ataques de suplantación de identidad o autorización indebida	0.52	4
RG-07	Aumento inesperado en la huella de carbono del sistema	0.25	3
RG-08	Falta de adopción por usuarios finales	0.48	2
RG-09	Errores en la medición de métricas ambientales	0.35	2
RG-10	Dependencia excesiva de proveedores externos	0.65	3

Tabla 23 - Análisis cuantitativo de riesgos

Fuente: Elaboración propia.

Trabajo Final de Grado – Prototipado Tecnológico

Se presenta el análisis cuantitativo y grado de exposición de los riesgos del proyecto, calculando el grado de exposición como el producto de la probabilidad de ocurrencia y el impacto, junto con el porcentaje y el porcentaje acumulado para aplicar el principio de Pareto. Esta tabla se ordena de mayor a menor grado de exposición, facilitando la identificación de los riesgos críticos en el contexto de la plataforma low-code con IA y enfoque eco-eficiente.

ID	Riesgo	Probabilidad de Ocurrencia	Impacto	Grado de Exposición	%	% Acumulado
RG-05	Vulnerabilidades de datos y fugas de información	0.8	4	3.2	18.09%	18.09%
RG-03	Fallos en la integración de IA con la plataforma low-code	0.75	4	3	16.96%	35.05%
RG-01	Retrasos en el cronograma de desarrollo	0.55	4	2.2	12.44%	47.49%
RG-06	Ataques de suplantación de identidad o autorización indebida	0.52	4	2.08	11.76%	59.25%
RG-10	Dependencia excesiva de proveedores externos	0.65	3	1.95	11.02%	70.27%
RG-04	Ineficiencia energética en aplicaciones generadas	0.5	3	1.5	8.48%	78.75%
RG-02	Sobrepaso del presupuesto asignado	0.45	3	1.35	7.63%	86.38%
RG-08	Falta de adopción por usuarios finales	0.48	2	0.96	5.43%	91.81%

RG-07	Aumento inesperado en la huella de carbono del sistema	0.25	3	0.75	4.24%	96.05%
RG-09	Errores en la medición de métricas ambientales	0.35	2	0.7	3.96%	100%

Tabla 24 - Análisis cuantitativo y grado de exposición

Fuente: Elaboración propia

A continuación, se realiza un diagrama de Pareto basado en los datos del análisis de riesgos del proyecto. Este diagrama se utiliza para visualizar la distribución de los riesgos según su grado de exposición, destacando los más críticos en el contexto de integración de IA y optimización ambiental. En otras palabras, se busca identificar el 20% de los riesgos que tienen la potencialidad de causar el 80% de los problemas en el proyecto.

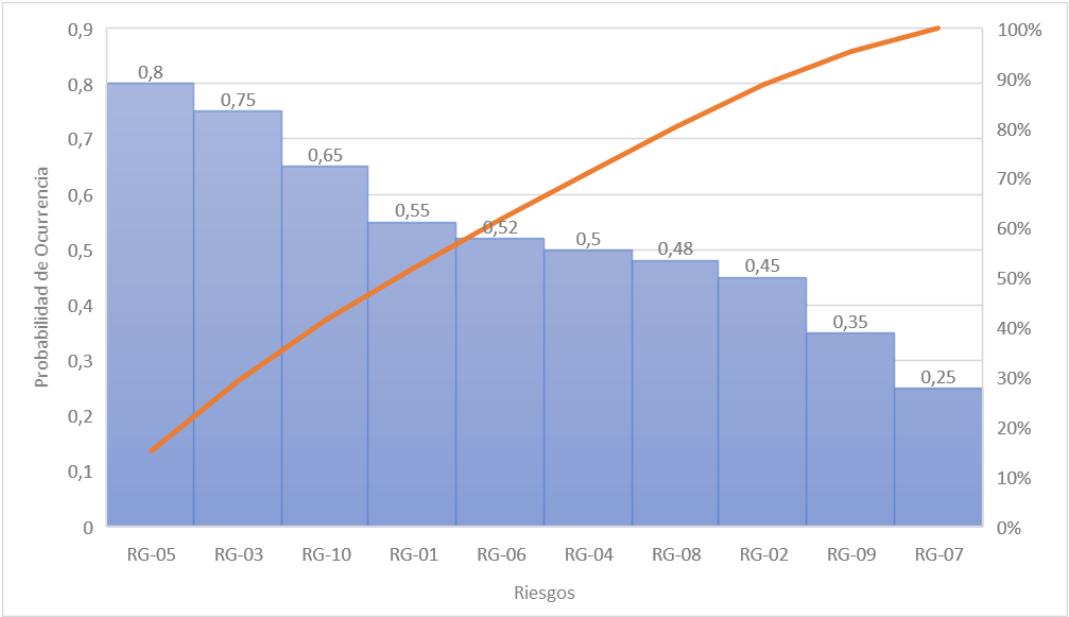


Figura 17 - Diagrama de Pareto

Fuente: Elaboración propia.

ID	Tipo	Riesgo	Plan de Contingencia
RG-05	De seguridad	Vulnerabilidades de datos y fugas de información	Se implementan auditorías de seguridad periódicas con pruebas de penetración para identificar y parchear vulnerabilidades en configuraciones low-code, junto con el uso de encriptación avanzada y monitoreo continuo de datos en AWS, activando protocolos de respuesta a incidentes.
RG-03	Técnico	Fallos en la integración de IA con la plataforma low-code	Se adopta un enfoque híbrido de desarrollo con pruebas iterativas en sprints SCRUM para validar integraciones de IA, incorporando soluciones de respaldo como APIs alternativas y automatización de pruebas de compatibilidad, con reasignación de recursos del equipo DevOps si se detectan incompatibilidades.
RG-01	Del proyecto	Retrasos en el cronograma de desarrollo	Se ajusta el backlog de SCRUM mediante refinamiento semanal y uso de burndown charts para monitorear avances, con activación de buffers temporales en sprints críticos y re-priorización de tareas no esenciales.
RG-06	De seguridad	Ataques de suplantación de identidad o autorización indebida	Se refuerza la autenticación con implementación obligatoria de MFA y detección de anomalías impulsada por IA para monitorear patrones de acceso, complementado con entrenamiento regular del equipo en ciberseguridad y protocolos de respuesta que incluyen bloqueo inmediato de cuentas sospechosas.
RG-10	Del proyecto	Dependencia excesiva de proveedores externos	Se diversifica proveedores mediante contratos con alternativas a AWS, (como Google Cloud), y adopción de estándares abiertos para facilitar migraciones, con revisiones periódicas.
RG-04	Técnico	Ineficiencia energética en	Se optimiza el código generado por IA mediante revisiones manuales y uso de

		aplicaciones generadas	herramientas como CodeCarbon para medir y refactorizar ineficiencias, integrando guías de prompts en el proceso de generación para priorizar eficiencia energética, con ajustes iterativos en sprints.
--	--	------------------------	--

Tabla 25 - Plan de contingencias

Fuente: Elaboración propia (Zenity, 2025; Mtex Tech, 2025).

Conclusiones

Este trabajo se centró en el diseño y prototipado de una plataforma low-code con inteligencia artificial, orientada a la medición y optimización de la huella de carbono en el desarrollo de software. Tras el desarrollo del prototipo y el análisis de sus componentes, se puede concluir que el objetivo general del proyecto se ha cumplido de manera satisfactoria, validando la viabilidad de la propuesta.

El prototipo tecnológico logró demostrar funcionalmente que es posible integrar herramientas de desarrollo rápido (entornos de trabajo low-code) con sistemas de inteligencia artificial y APIs de medición ambiental (como CodeCarbon y Climatiq). Se validó el core del sistema, materializado en un dashboard que unifica métricas de consumo energético y estimaciones de CO2 en tiempo real. Asimismo, se demostró la capacidad de la IA para analizar el código y proponer refactorizaciones automáticas que reducen la ineficiencia. De esta forma, el prototipo da una respuesta concreta a la problemática de la dispersión de métricas y la falta de optimización sostenible en los procesos de desarrollo tradicionales. Si bien la meta de una reducción del 70% en el consumo permanece como una proyección teórica validada por la bibliografía, el prototipo sienta las bases funcionales y técnicas para alcanzar dicho objetivo en una implementación completa.

Basado en los hallazgos de este proyecto, se emiten las siguientes recomendaciones para la industria del software:

- Se recomienda incorporar la medición de la huella de carbono como un requisito no funcional explícito en las etapas tempranas (relevamiento y diseño) de todo nuevo proyecto de software.
- Se sugiere auditar y validar las fuentes de datos de las APIs de medición ambiental. Como se identificó en el análisis de riesgos (RG-09), la precisión de las

métricas de ecoeficiencia depende críticamente de la actualización de los datos regionales sobre la matriz energética.

- Es recomendable implementar planes de capacitación para los equipos de desarrollo. El análisis de riesgos (RG-08) identificó la resistencia al cambio y la falta de alineación con estas nuevas métricas ecoeficientes como un riesgo de gestión significativo para la adopción de estas tecnologías.

Dado que este proyecto consiste en un prototipo funcional, se abren claras líneas de trabajo a futuro para evolucionar esta iniciativa hacia un producto completo:

- Validación en entornos reales: Se propone realizar una prueba piloto de la plataforma en una organización real, (superando la organización modelada), para medir empíricamente la reducción de emisiones y obtener retroalimentación directa de equipos de desarrollo.

- Expansión del módulo de IA: Como trabajo futuro, se podría entrenar modelos de IA más avanzados que no solo sugieran refactorizaciones, sino que las apliquen de forma autónoma (con supervisión humana) y aprendan de los patrones de consumo específicos de la infraestructura del cliente.

- Integración con CI/CD: Finalmente, se recomienda desarrollar módulos que integren las métricas de huella de carbono directamente en los pipelines de Integración Continua y Despliegue Continuo (CI/CD), permitiendo bloquear automáticamente compilaciones o despliegues que superen un umbral de emisiones predefinido.

Prototipo

En el siguiente enlace, se puede descargar el código fuente del prototipo.

Link: <https://github.com/clrecce/TFG>

Referencias

AFP. (2025). El impacto medioambiental de la Inteligencia Artificial en cinco cifras. Ojo al clima. Recuperado de <https://ojoalclima.com/articles/el-impacto-medioambiental-de-la-inteligencia-artificial-en-cinco-cifras>

Amazon Web Services. (2025a). *Nube híbrida con AWS*. Recuperado de <https://aws.amazon.com/es/hybrid/>

Amazon Web Services. (2025b). *Infraestructura sostenible de AWS*. Recuperado de <https://aws.amazon.com/es/sustainability/data-centers/>

Antonio Romero y Craig Hale. (2024). El consumo energético de los centros de datos se está convirtiendo en un grave problema. *Techradar*. Recuperado de <https://global.techradar.com/es-es/pro/el-consumo-energetico-de-los-centros-de-datos-se-esta-convirtiendo-en-un-grave-problema>

Appsmith. (2025). *18 prácticas recomendadas para desarrollar aplicaciones web seguras y de alto rendimiento con poco código*. Recuperado de <https://www.appsmith.com/blog/low-code-buyers-guide-best-practices>

Appsmith. (2025). *Precios de Appsmith*. Recuperado de <https://www.appsmith.com/pricing>

Atlassian. (s.f.). *Product backlog: consejos para su creación y priorización*. Recuperado de <https://www.atlassian.com/agile/scrum/backlogs>

AWS. (2025). *Precios de AWS*. Recuperado de <https://aws.amazon.com/es/s3/pricing/>

BBVA. (2023). *¿Qué es el ‘green software’ y cuáles son sus beneficios?* Recuperado de <https://www.bbva.com/es/innovacion/que-es-el-green-software-y-cuales-son-sus-beneficios/>

Bioscore. (2024). *Green coding: The environmental footprint of programming languages and frameworks*. Recuperado de <https://bioscore.com/blog/green-coding-the-environmental-footprint-of-programming-languages-and-frameworks>

Banco de la Nación Argentina (BNA). (2025, 24 de octubre). *Cotización dólar BNA*. Recuperado de <https://www.bna.com.ar/Personas>

Bubble. (2025). *Lanzar aplicaciones, no prototipos*. Recuperado de <https://bubble.io/>

CIO. (2024). Los copilotos de IA generativa revolucionan el 'low code' para acelerar el 'time-to-market' y minimizar costes del desarrollo de 'software'. *CIO*. Recuperado de <https://www.cio.com/article/3616160/los-copilotos-de-ia-generativa-revolucionan-el-low-code-para-acelerar-el-time-to-market-y-minimizar-costes-del-desarrollo-de-software.html>

Climatiq. (2023). *Climatiq API documentation*. Recuperado de <https://api.climatiq.io>

Climatiq. (2025). *Precios de Climatiq*. Recuperado de <https://www.climatiq.io/pricing>

Cloudwards. (2025). *Estadísticas de sostenibilidad de la nube: definición y esfuerzos en 2025*. Recuperado de <https://www.cloudwards.net/cloud-sustainability-statistics/>

Cloudwards. (2025). *Precios de cloudwards*. Recuperado de <https://www.cloudwards.net/cloud-sustainability-statistics/>

CodeCarbon. (2025). *Realice un seguimiento y reduzca las emisiones de CO2 de su informática*. Recuperado de <https://codecarbon.io/>

Codector. (s.f.). *Low code, la evolución del desarrollo*. Recuperado de <https://codector.com.co/low-code-la-evolucion-del-desarrollo/>

Cogent Info. (2025). *Computación verde en la nube: estrategias de infraestructura de TI sostenible*. Recuperado de <https://www.cogentinfo.com/resources/green-computing-in-the-cloud-sustainable-it-infrastructure-strategies>

Consejo Profesional de Ciencias Informáticas de Buenos Aires (CPCIBA). (2025). *Tabla de referencia de honorarios Actualización Agosto 2025*. Recuperado de <https://plataforma.cpciba.org.ar/autogestion/honorarios#>

Crownhart, J. O. (2025). Calculamos el impacto energético de la IA. Aquí está la historia que no conocías. *Technology Review*. Recuperado de

<https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/>

E-SPIN Group. (2025). *Green Coding*. Recuperado de <https://www.e-spincorp.com/green-coding-energy-efficient-software/>

Fernanda González. (2024). Las emisiones contaminantes de Google se disparan a causa de la IA. *Wired*. Recuperado de <https://es.wired.com/articulos/las-emisiones-contaminantes-de-google-se-disparan-a-causa-de-la-ia>

Footprint, C. C. (s.f.). *Huella de carbono en la nube*. Recuperado de <https://www.cloudcarbonfootprint.org/docs/>

Frontegg. (2025). *¿Qué es la autenticación de usuario? Una guía para aplicaciones modernas*. Recuperado de <https://frontegg.com/blog/authentication>

Gartner. (2025). *Cuadrante Mágico de Gartner para Plataformas de Aplicaciones Empresariales de Bajo Código*. Recuperado de <https://www.gartner.com/en/documents/6773234>

Gautam IT Services. (2025). *Cómo el Low-Code/No-Code es sostenible: Reducción de carbono mediante software*. Recuperado de <https://www.google.com/search?q=https://www.gautamitservices.com/blogs/how-low-codeno-code-is-sustainable-reducing-carbon-through-software>

Georgiou, S. K. (2019). *Análisis del consumo energético de los lenguajes de programación: Un estudio empírico*. Recuperado de https://stefanos1316.github.io/my_curriculum_vitae/GKS17.pdf

Green Software Foundation. (2023). *Introducción | Green software*. Recuperado de <https://learn.greensoftware.foundation/introduction/>

Grupo Vansur. (2025). *El Costo Real del Software a Medida en Argentina*. Recuperado de <https://grupovansur.com/el-costo-real-del-software-a-medida-en-argentina-2025-una-guia-transparente/>

Hivenet. (2025). *Cloud Backup: Complete Guide to Secure Data Protection*. Recuperado de <https://www.hivenet.com/post/cloud-backup-complete-guide-to-secure-data-protection-in-2025>

Holdfast Projects. (2025). *¿Conocen las IA sus emisiones de carbono? No.* Recuperado de <https://www.holdfastprojects.com/chat-carbon-emissions>

IBM. (2023). *Green coding.* Recuperado de <https://www.ibm.com/think/topics/green-coding>

Ideas2IT. (2025). *La IA en el ciclo de vida del desarrollo de software.* Recuperado de <https://www.ideas2it.com/blogs/agent-ai-agile-software-delivery>

Infrascale. (2025). *Data Backup Solutions.* Recuperado de <https://www.infrascale.com/data-backup-solutions-statistics-usa/>

Jens Gröger, F. B. (2025). *Impactos ambientales de la inteligencia artificial.* Greenpeace.

Jitterbit. (2025). *El futuro del low-code.* Recuperado de <https://www.jitterbit.com/blog/the-future-of-low-code/>

Kodffe. (2025). *Future software development technological trends 2025.* Recuperado de https://www.kodffe.com/article/future-software-development-technological-trends-2025?language_content_entity=es&language=es

Kolla. (2025). *Por qué es crucial actualizar periódicamente sus imágenes Kolla en OpenStack.* Recuperado de <https://openmetal.io/resources/blog/why-its-crucial-to-regularly-update-your-kolla-images-on-openstack/>

Legit Security. (2025). *Generación de código con IA: riesgos y beneficios de la IA en el software.* Recuperado de <https://www.legitsecurity.com/aspm-knowledge-base/ai-code-generation-benefits-and-risks>

Lottick, K. S. (2019). *Realice un seguimiento y reduzca las emisiones de CO2 de su informática.* Recuperado de <https://codecarbon.io/>

Mediaware. (2025, 25 de abril). Los Data Centers representan casi el 2% del consumo eléctrico mundial. *ITWARE.* Recuperado de <https://www.itwarelatam.com/2025/04/25/los-data-centers-representan-casi-el-2-del-consumo-electrico-mundial/>

Meghan Rimol DeLisi y Catherine Howley. (2022). Gartner prevé que el mercado mundial de tecnologías de desarrollo low-code crecerá un 20% en 2023. *Gartner News.*

Recuperado de <https://www.gartner.com/en/newsroom/press-releases/2022-12-13-gartner-forecasts-worldwide-low-code-development-technologies-market-to-grow-20-percent-in-2023>

Mendix. (2025). *Mendix: Low-Code Application Development Platform*. Recuperado de <https://www.mendix.com/>

Mendix. (2025). *Precios de Mendix*. Recuperado de <https://www.mendix.com/pricing/>

Microsoft. (2025). *Microsoft Power Apps*. Recuperado de <https://www.microsoft.com/en-us/power-platform/products/power-apps>

Microsoft. (2025). *Precios de Power Apps*. Recuperado de <https://www.microsoft.com/en-us/power-platform/products/power-apps/pricing/>

Mtex Tech. (2025). *IA sostenible: promoción de prácticas ecológicas en el desarrollo tecnológico*. Recuperado de <https://mtextech.com/sustainable-ai-promoting-eco-friendly-practices-in-ai/>

News Center Microsoft Latinoamérica. (2025). *Nuestro Informe de Sostenibilidad Ambiental 2025. Microsoft Latinoamérica*. Recuperado de <https://news.microsoft.com/es-xl/nuestro-informe-de-sostenibilidad-ambiental-2025/>

OutSystems. (2025). *Precios de OutSystems*. Recuperado de <https://www.outsystems.com/pricing-and-editions/>

Outsystems. (2025a). *La plataforma líder de desarrollo de código bajo impulsada por IA*. Recuperado de <https://www.outsystems.com/low-code-platform/>

Outsystems. (2025b). *Creador de aplicaciones de drag-and-drop: Impulsando la eficiencia del desarrollo*. Recuperado de <https://www.outsystems.com/application-development/app-builders/drag-and-drop-app-builder/>

OWASP. (2025). *Authentication Cheat Sheet*. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Paull Young. (2024). *Las 10 mejores herramientas para hacer más ecológico tu software. GitHub Blog*. Recuperado de <https://github.blog/open-source/social-impact/the-10-best-tools-to-green-your-software/>

Pereira, R. F. (2020). *Prácticas de codificación ecológica en el desarrollo de software: Percepciones, barreras y estrategias para el cambio*. (Tesis de Maestría). DiVA. Recuperado de <https://www.diva-portal.org/smash/get/diva2:1909766/FULLTEXT01.pdf>

Piattini, C. (2015). *Green in software engineering*.

Poidevin, O. L. (2025). Las emisiones indirectas de los gigantes tecnológicos aumentaron un 150% en tres años a medida que se expande la IA, dice agencia de la ONU. *Reuters*. Recuperado de <https://www.reuters.com/sustainability/climate-energy/tech-giants-indirect-emissions-rose-150-three-years-ai-expands-un-agency-says-2025-06-05/>

Processa. (2025). *Los 10 mejores programas de bajo código/sin código de OWASP*. Recuperado de <https://owasp.org/www-project-top-10-low-code-no-code-security-risks/>

Rubrik. (2025). *¿Qué es la copia de seguridad en la nube?* Recuperado de <https://www.rubrik.com/insights/what-is-cloud-backup>

S&P Global Market Intelligence. (2018). *Low code statistics*. Recuperado de <https://www.appbuilder.dev/es/low-code-statistics>

Scale Computing. (2025). *Cómo garantizar una alta disponibilidad en entornos de TI distribuidos*. Recuperado de <https://www.scalecomputing.com/resources/high-availability-in-distributed-it-environments>

Scrum Institute. (2025). *The Scrum product backlog*. Recuperado de https://www.scrum-institute.org/The_Scrum_Product_Backlog.php

SeaAsia Infotech. (2025). *Cómo el desarrollo low-code/no-code puede reducir la huella de carbono de tu aplicación*. Recuperado de <https://www.seasiainfotech.com/blog/low-code-no-code-development-for-carbon-reduction/>

Seidor. (2025). *La IA como impulsora de la sostenibilidad en las empresas*.

SISA. (2025). *10 Cybersecurity Best Practices in the Age of AI*. Recuperado de <https://www.sisainfosec.com/blogs/10-cybersecurity-best-practices-in-the-age-of-ai-2025/>

Solar Tech Online. (2025). *Software de eficiencia energética: Guía completa para elegir la solución adecuada para su negocio*. Recuperado de <https://solartechonline.com/blog/energy-efficiency-software-guide/>

Superblocks Team. (2025). *Low code use cases*. Recuperado de <https://www.superblocks.com/blog/low-code-use-cases>

Techtalent. (2025). *Green software development in 2025: Cutting costs and carbon in the digital era*. Recuperado de <https://www.techtalent.ro/green-software-development-in-2025-cutting-costs-and-carbon-in-the-digital-era/>

The AI Journal. (2025). La paradoja oculta del desarrollo low-code. *The AI Journal*. Recuperado de <https://aijourn.com/the-hidden-paradox-of-low-code-development-navigating-the-tightrope-between-speed-and-sustainability/>

The International Telecommunication Union. (2025). *Greening Digital Companies: Monitoring emissions and climate commitments report 2025*. Recuperado de <https://www.itu.int/es/mediacentre/Pages/PR-2025-06-05-greening-digital-companies-report.aspx>

TsTTechnology. (2025). *Software Products Generate 85% Less CO2 Than High-Emission Sectors*. Recuperado de <https://tsstechnology.io/blog/software-products-85-percent-less-co2>

Ugarte Dessire. (2024). Las 5 mejores plataformas de IA de bajo código para crear aplicaciones empresariales inteligentes en 2025. *Appsmith*. Recuperado de <https://www.appsmith.com/blog/top-low-code-ai-platforms>

Unitrends. (2025). *The State of Backup and Recovery Report*. Recuperado de <https://www.unitrends.com/media/downloads/resources/The-State-of-Backup-and-Recovery-Report-2025.pdf>

Wiz. (2025). *Mejores prácticas de seguridad en la nube para 2025*. Recuperado de <https://www.wiz.io/academy/cloud-security-best-practices>

Zenity. (2025). *Mejores prácticas y estrategias de seguridad de aplicaciones de código bajo*. Recuperado de <https://zenity.io/blog/security/low-code-application-security-best-practices-and-strategies>

Zennobi. (2025). *Impulse su desarrollo con IA y plataformas low code*. Recuperado de <https://zennobi.com/ia/impulse-su-desarrollo-con-ia-y-plataformas-low-code/>

Zmanda. (2025). *¿Qué es la política de respaldo de datos?* Recuperado de <https://www.zmanda.com/blog/what-is-data-backup-policy/>

Anexos

Anexo 1.1: Estructura de entrevista para Gerente de Desarrollo de Software

- Objetivo: Recopilar información sobre los procesos de desarrollo de software existentes en la empresa modelada (EcoDev Tech), identificar ineficiencias energéticas y comprender la percepción de los roles clave sobre la sostenibilidad en el software.

- Metodología: Entrevistas semiestructuradas realizadas a roles clave involucrados en el ciclo de vida del desarrollo.

- Tema: Gestión de Proyectos y Recursos

- Entrevistado 1: Gerente de Desarrollo de Software

Pregunta 1- Flujo de Trabajo: ¿Podrías describir el proceso actual de desarrollo de software, desde la idea hasta el despliegue?

Pregunta 2 - Planificación: Durante la fase de planificación de un proyecto, ¿se considera de alguna forma el consumo de recursos computacionales (CPU, memoria) que tendrá la aplicación?

Pregunta 3 - Métricas: ¿Disponen actualmente de algún dashboard o reporte que centralice el consumo energético de las aplicaciones en producción?

Pregunta 4 - Ineficiencias: ¿Cuáles son los cuellos de botella más comunes en sus ciclos de desarrollo? ¿Cuánto tiempo se dedica a la optimización de código después del despliegue?

Pregunta 5 - Sostenibilidad: ¿Existe alguna iniciativa o política en la empresa relacionada con "Green Coding" o la reducción de la huella de carbono digital?

Pregunta 6 - Herramientas: ¿Qué herramientas utilizan para la gestión de requisitos y el monitoreo de rendimiento? ¿Están integradas?

Pregunta 7 - Visión a Futuro: ¿Qué tan valioso consideraría tener una plataforma que sugiera optimizaciones de código en tiempo real para reducir el consumo energético?

Anexo 1.2: Estructura de entrevista para Desarrollador Senior

- Tema: Prácticas de Codificación y Optimización
- Entrevistado 2: Desarrollador Senior

Pregunta 1 - Codificación Diaria: En tu día a día, ¿cuáles son los principales desafíos al escribir código? (Ej. plazos, complejidad).

Pregunta 2 - Optimización: ¿Con qué frecuencia realizas refactorización de código enfocada en mejorar el rendimiento (ej. reducir uso de CPU o memoria)? ¿Es una tarea prioritaria?

Pregunta 3 - Identificación de Problemas: ¿Cómo identificas actualmente un fragmento de código "ineficiente" desde el punto de vista energético? ¿Es por intuición, por reportes de profiling, o por quejas de lentitud?

Pregunta 4 - Herramientas de Monitoreo: ¿Qué herramientas usas para medir el rendimiento de tu código localmente o en entornos de prueba? ¿Miden consumo energético o solo tiempo de ejecución?

Pregunta 5 - Métricas Ambientales: ¿Alguna vez has intentado calcular o estimar las emisiones de CO2 de una aplicación en la que trabajas? Si no, ¿por qué?

Pregunta 6 - "Green Coding": ¿Estás familiarizado con el término "Green Coding"? ¿Conoces prácticas específicas (ej. optimizar bucles, elegir algoritmos eficientes) con ese fin?

Pregunta 7 - Automatización (IA/Low-Code): ¿Estarías dispuesto a usar una herramienta de IA que analice tu código y te sugiera refactorizaciones automáticas para hacerlo más eco-eficiente? ¿Qué preocupaciones tendrías?

Anexo 2: Hoja de Requisitos Inicial – Ejemplo ficticio

- Descripción: Ejemplo de la plantilla de relevamiento de requisitos (formato tipo Excel) utilizada por EcoDev Tech antes de la implementación del prototipo. Destaca la ausencia total de métricas de sostenibilidad o eficiencia energética.
- Proyecto: Sistema de Gestión de Clientes (CRM)
- Fecha: 15 de octubre de 2025
- Responsable: Analista de Negocios

ID	Módulo	Descripción Funcional	Prioridad	Estado	Observaciones
REQ-001	Autenticación	El usuario debe poder iniciar sesión con email y contraseña.	Alta	Pendiente	Incluir "Recordar contraseña".
REQ-002	Autenticación	El sistema debe permitir el registro de nuevos usuarios.	Alta	Pendiente	Validar que el email sea único.
REQ-003	Dashboard	El usuario debe ver un resumen de sus clientes y tareas pendientes al iniciar sesión.	Media	Pendiente	
REQ-004	Clientes	Permitir la creación, edición y eliminación de clientes (CRUD).	Alta	Pendiente	Campos: Nombre, Apellido, Email, Teléfono.

Trabajo Final de Grado – Prototipado Tecnológico

REQ-005	Reportes	El sistema debe generar un reporte mensual de clientes nuevos en formato PDF.	Media	Pendiente	
REQ-006	Reportes	El sistema debe generar un reporte de actividad de usuarios.	Baja	Pendiente	

Anexo 3: Reporte de Pruebas de QA – Ejemplo ficticio

- Descripción: Formato de reporte de QA estándar, enfocado únicamente en la validación funcional (Éxito/Fallo) y errores detectados. No incluye ninguna métrica de rendimiento de recursos ni de consumo energético durante las pruebas.
- Proyecto: Sistema de Gestión de Clientes (CRM)
- Fecha: 27 de octubre de 2025
- Responsable de QA: Analista de QA
- Resumen de Ejecución:
 - o Casos de Prueba Ejecutados: 15
 - o Resultados Éxito: 11
 - o Resultados Fallo: 4
 - o Bugs Reportados: 4
- Detalle de Casos de Prueba (Extracto):

ID	Descripción del Caso	Pasos de Prueba	Resultado Esperado	Resultado Obtenido	Estado	Observaciones
QA-001	Login exitoso	1. Ir a /login. 2. Ingresar 'user@test.com'. 3. Ingresar 'Pass123!'. 4. Clic en "Ingresar".	Redirección al Dashboard.	Redirección al Dashboard.	Éxito	

Trabajo Final de Grado – Prototipado Tecnológico

QA-002	Login fallido (pass incorrecta)	1. Ir a /login. 2. Ingresar 'user@test.com'. 3. Ingresar 'pass-Mala'. 4. Clic en "Ingresar".	Mensaje "Credenciales inválidas".	Mensaje "Credenciales inválidas".	Éxito	
QA-003	Registro de nuevo usuario	1. Ir a /registro. 2. Completar formulario 3. Clic en "Registrar".	Usuario creado y redirección a /login.	Error 500 del servidor.	Fallo	Bug #1024 reportado. La API de registro falla.
QA-015	Generar reporte PDF	1. Ir a /reportes. 2. Clic en "Generar PDF".	Descarga de reporte.pdf.	El navegador se congela por 30 segundos y luego descarga el PDF.	Fallo	Bug #1025 reportado. El reporte es funcional pero inaceptablemente lento.

Anexo 4: Log de Monitoreo de Recursos – Ejemplo ficticio

- Descripción: Ejemplo de un archivo log.txt generado por las herramientas de monitoreo de infraestructura actuales. Las métricas (CPU, memoria) están dispersas, no correlacionadas por aplicación y carecen de contexto sobre el impacto ambiental.

```
[2025-08-12 10:30:01] INFO: service:api-auth-svc status:running
[2025-08-12 10:30:15] WARN: server:srv-app-01 metric:CPU_Usage value:75%
[2025-08-12 10:30:15] INFO: server:srv-app-01 metric:MEM_Usage
value:6.8GB/8GB
[2025-08-12 10:30:45] WARN: server:srv-app-01 metric:CPU_Usage value:78%
[2025-08-12 10:31:02] INFO: service:api-report-gen status:starting
[2025-08-12 10:31:15] WARN: server:srv-app-01 metric:CPU_Usage value:95%
[2025-08-12 10:31:15] INFO: server:srv-app-01 metric:MEM_Usage
value:7.5GB/8GB
[2025-08-12 10:31:30] ERROR: service:api-report-gen status:crashed
reason:OOM (Out of Memory)
[2025-08-12 10:31:35] WARN: server:srv-app-01 metric:CPU_Usage value:40%
[2025-08-12 10:31:35] INFO: server:srv-app-01 metric:MEM_Usage
value:3.1GB/8GB
[2025-08-12 10:32:00] INFO: service:api-auth-svc status:running
[2025-08-12 10:33:00] WARN: server:srv-db-01 metric:Disk_IO_Wait value:35%
```

Anexo 5: Estimación de CO2 Manual – Ejemplo ficticio

- Descripción: Ejemplo de la hoja de cálculo (formato Excel) utilizada "ad hoc" por el equipo de operaciones para estimar la huella de carbono de la infraestructura. Los datos son manuales, agregados (no por aplicación) y se basan en herramientas externas.

Componente	Proveedor	Consumo Eléctrico Estimado (kWh)	Factor de Emisión (kg CO2e/kWh)	Emisiones Totales (kg CO2e)	Notas
Servidores Locales (On-premise)	EcoDev Tech (Data Center Propio)	1,200 kWh	0.43 (Promedio Red Eléctrica Argentina)	516 kg	Estimación basada en factura eléctrica total del rack.
Instancias Cloud (App-Servers)	AWS (us-east-1)	850 kWh	0.39 (Factor de Emisión AWS para esa región)	331.5 kg	Datos obtenidos manualmente de AWS Cost Explorer.
Instancias Cloud (Base de Datos)	AWS (us-east-1)	400 kWh	0.39 (Factor de Emisión AWS para esa región)	156 kg	Datos obtenidos manualmente de AWS Cost Explorer.

Trabajo Final de Grado – Prototipado Tecnológico

Total Estimado		2,450 kWh		1,003.5 kg	Reporte generado manualmente. No hay desglose por aplicación.
-------------------	--	--------------	--	------------	--